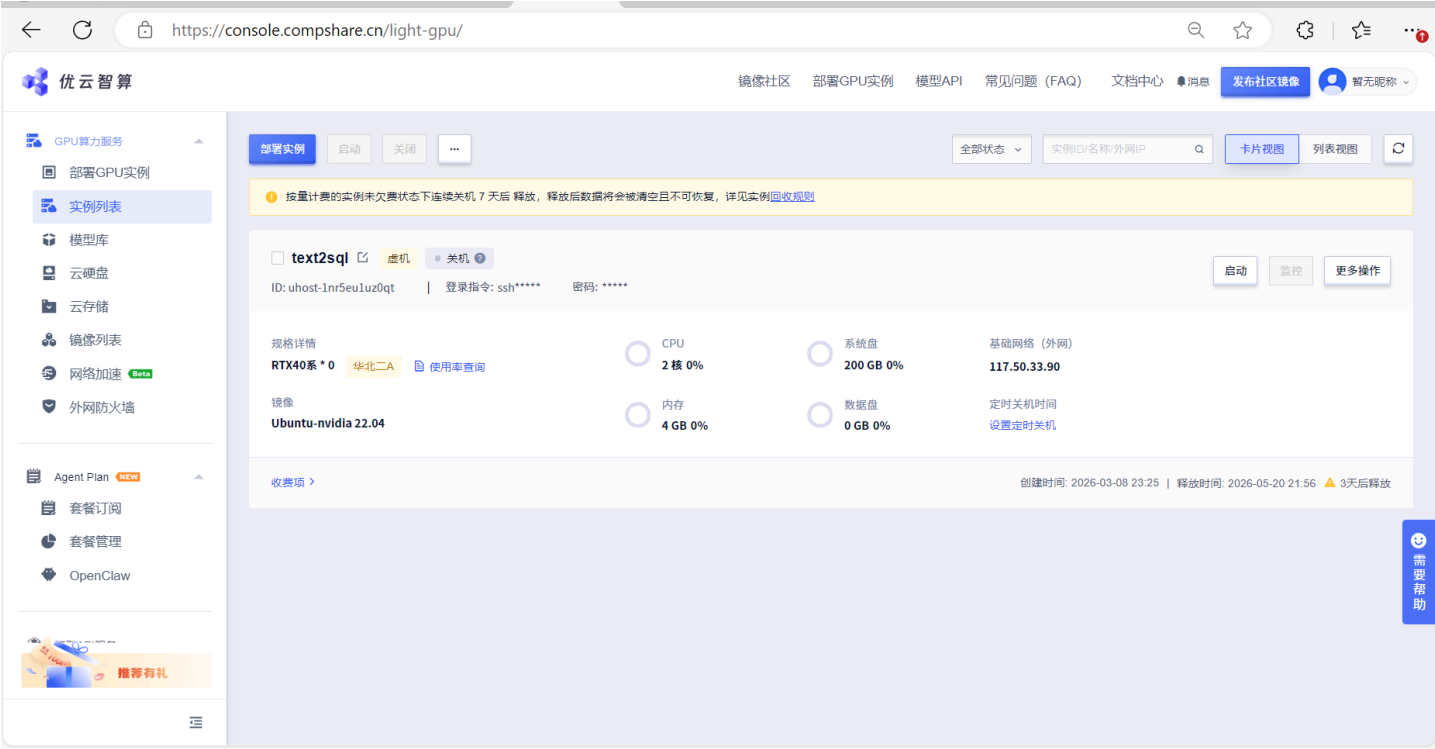


企业智能问数系统

1. 智能问数据项目环境搭建

2.1 服务器租赁

服务器租赁平台：<https://www.compshare.cn/>

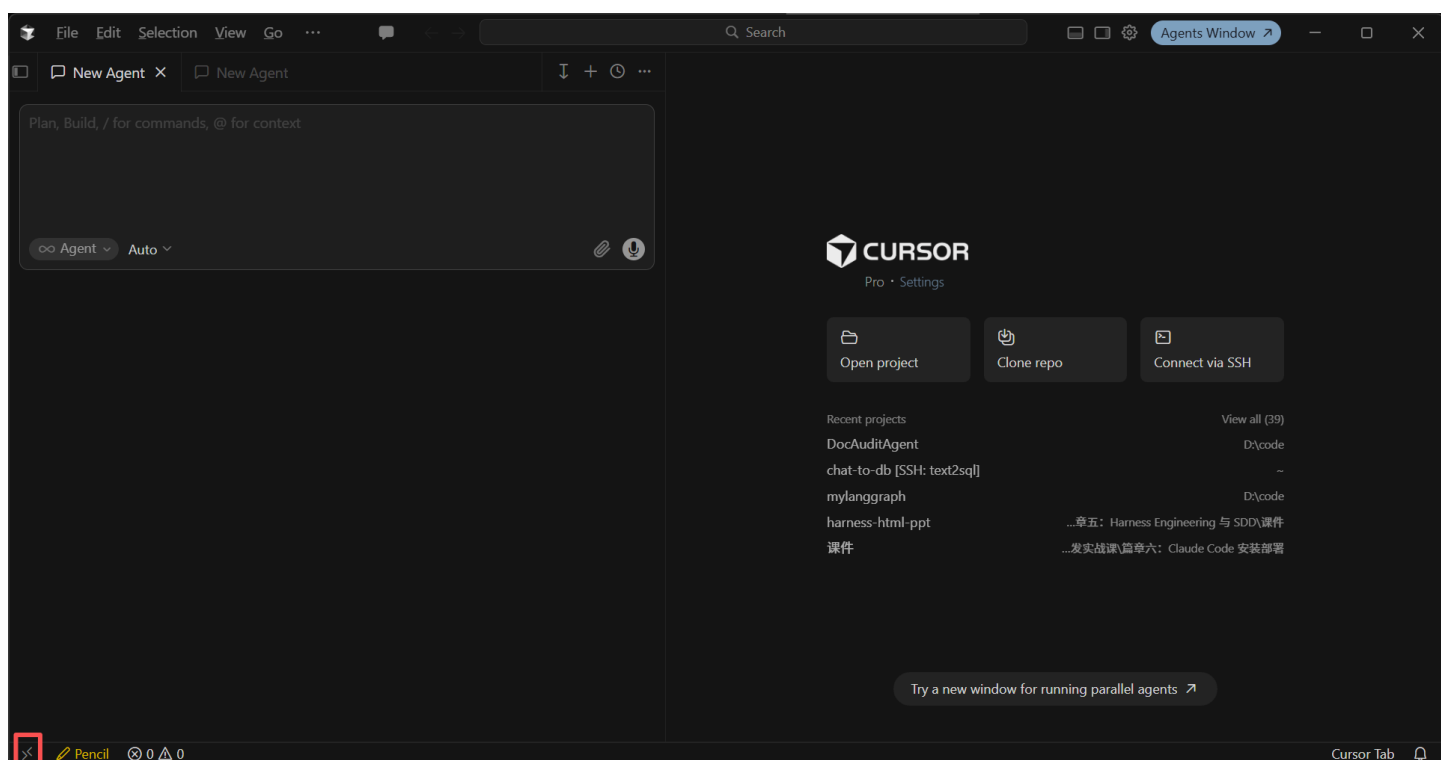


2.2 开发工具安装

Cursor官网：<https://cursor.com/cn>



2.3 开发环境配置



2. 总体设计和技术选型

后台系统

前端：ReAct

后端：FastAPI

知识图谱：Neo4j

RAG流程（样本训练）：DeepSeek, qwen3-embedding, Milvus

前台系统（Text2SQL,ChatBI)

前端：ReAct

后端：FastAPI

大模型：DeepSeek

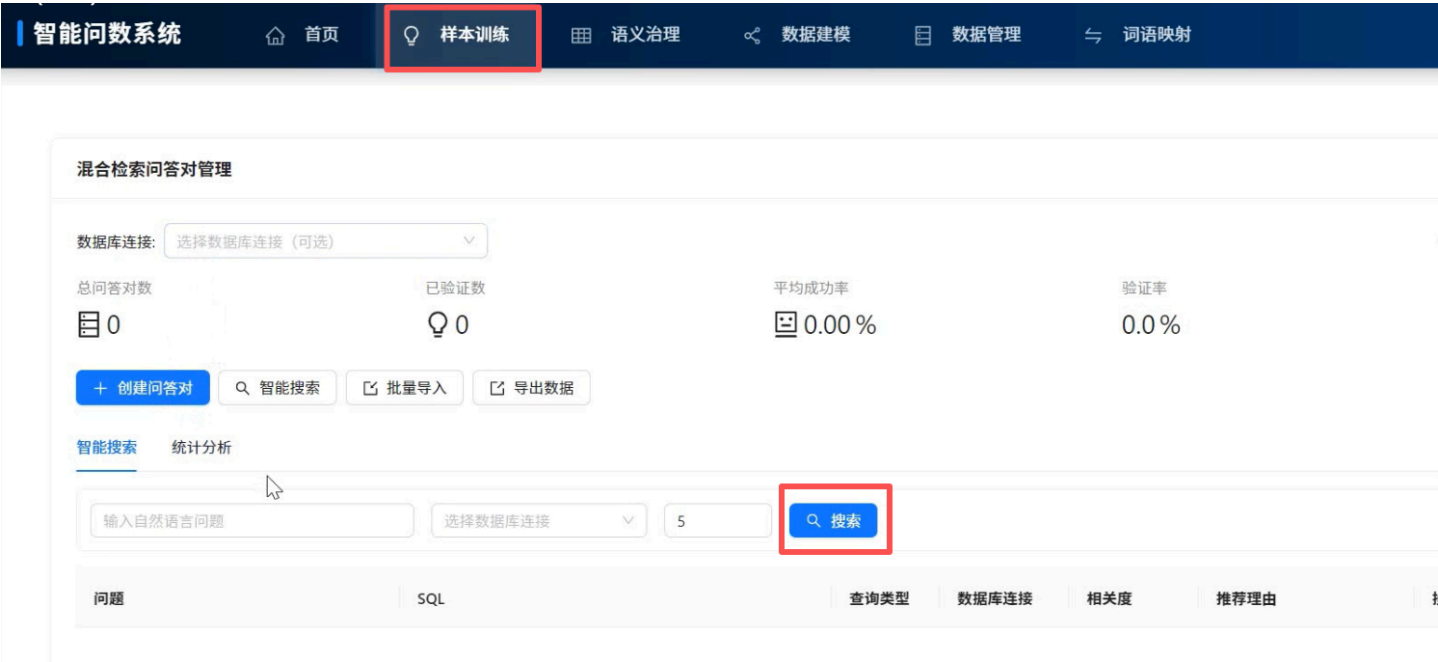
大模型开发框架：LangChain，LangGraph（Multi Agent，MCP）

开发工具：Cursor/Pycharm/Claude Code

Vibe Coding 课程链接：<https://www.bilibili.com/cheese/play/ss905865514>

3. 智能问数据系统部署

1.1 Ollama服务启动



地址：<https://ollama.com/>，安装部署Ollama

代码块

```
1 curl -fsSL https://ollama.com/install.sh | sh
```

代码块

```
1 ollama pull qwen3-embedding
```

代码块

```
1  ## 验证命令
2  ollama list
```

1.2 数据库启动

安装MySQL	<pre>sudo apt-get update sudo apt-get install mysql-server sudo service mysql start</pre>
创建账户	<pre>CREATE USER 'gpt'@'localhost' IDENTIFIED BY 'gpt';</pre>
给账户赋权限	<pre>GRANT ALL PRIVILEGES ON *.* TO 'gpt'@'localhost'; FLUSH PRIVILEGES;</pre>
创建数据库	<pre>create database chatdb;</pre>

代码块

```
1  #启动命令
2  sudo service mysql start
3  #验证命令
4  sudo service mysql status
```

1.3 Milvus向量数据库启动

代码块

```
1  conda create -n milvus python=3.12
2  conda activate milvus
3
4  mkdir -p ~/milvus && cd ~/milvus
5
```

代码块

```
1  # 更新软件源
2  sudo apt update && sudo apt upgrade -y
3
4  # 安装 Docker 依赖
```

```
5  sudo apt install -y apt-transport-https ca-certificates curl software-  
   properties-common  
6  
7  # 添加 Docker GPG 密钥  
8  curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -  
   o /usr/share/keyrings/docker-archive-keyring.gpg  
9  
10 # 添加 Docker 源  
11 echo "deb [arch=$(dpkg --print-architecture) signed-  
   by=/usr/share/keyrings/docker-archive-keyring.gpg]  
   https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee  
   /etc/apt/sources.list.d/docker.list > /dev/null  
12  
13 # 安装 Docker  
14 sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io  
15  
16 # 安装 Docker Compose (v2+)  
17 sudo apt install -y docker-compose-plugin  
18  
19 # 验证安装  
20 docker --version          # 输出 Docker 版本 (如 26.0.0)  
21 docker compose version    # 输出 Compose 版本 (如 v2.25.0)  
22  
23 # 非 root 用户添加 Docker 权限 (避免每次加 sudo)  
24 sudo usermod -aG docker $USER  
25 # 生效权限 (需重新登录服务器)  
26 newgrp docker  
27  
28 ##安装Milvus  
29 # 创建目录并进入  
30 mkdir -p ~/milvus && cd ~/milvus  
31 # 下载 Milvus 2.4.x 最新版配置 (兼容大部分场景)  
32 wget https://github.com/milvus-io/milvus/releases/download/v2.4.5/milvus-  
   standalone-docker-compose.yml -O docker-compose.yml  
33  
34 # 可选: 查看配置文件 (确认端口/路径)  
35 cat docker-compose.yml  
36 # 启动 Milvus (后台运行)  
37 sudo docker compose up -d  
38  
39 # 查看容器状态 (需等待所有容器启动, 约 1-2 分钟)  
40 docker compose ps
```

代码块

```
1  # 1. 创建 Docker 配置目录
```

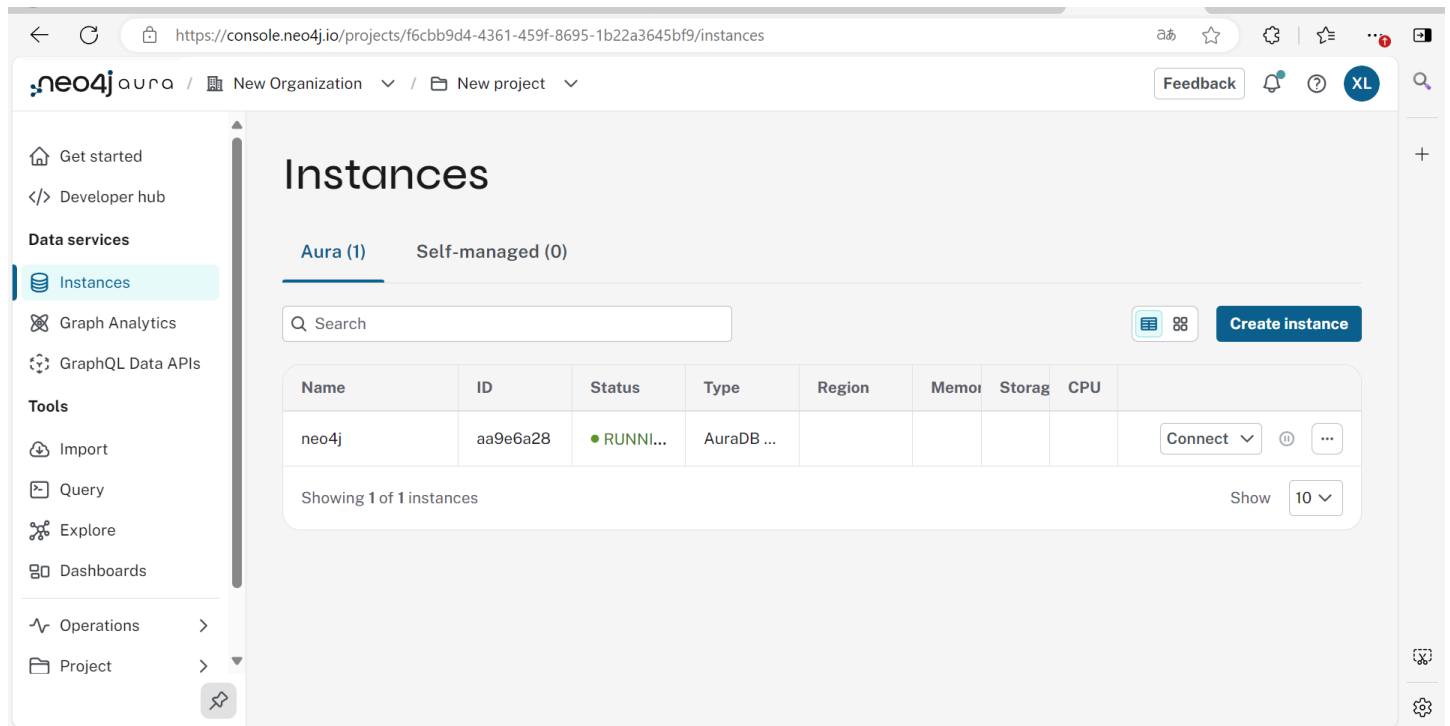
```
2  sudo mkdir -p /etc/docker
3
4  # 2. 编辑 daemon.json 配置文件
5  sudo vi /etc/docker/daemon.json
6
7  {
8      "registry-mirrors": [
9          "https://docker.m.daocloud.io",
10         "https://hub-mirror.c.163.com",
11         "https://mirror.ccs.tencentyun.com"
12     ]
13 }
14
15
16 # 3. 重启 Docker 服务
17 sudo systemctl daemon-reload
18 sudo systemctl restart docker
19 # 4. 重新拉取 Milvus 镜像 (先清理失败的容器)
20 sudo docker compose down sudo docker compose pull
21 # 先拉取镜像
22 sudo docker compose up -d
```

代码块

```
1  ## 进去虚拟环境
2  conda activate milvus
3  ## 进入命令
4  cd /home/ubuntu/milvus
5  ## docker启动
6  sudo docker compose up -d
```

1.4 图数据库安装部署(Neo4j)

地址: <https://neo4j.com/>



1.5 后台系统后端服务启动

虚拟环境构建以及依赖安装

代码块

```
1 conda create -n sql python=3.12
2 conda activate sql
3 pip install -r requirements.txt
```

代码块

```
1 ## 进入命令
2 cd /home/ubuntu/chat-to-db/backend
3 ## 切入虚拟环境
4 conda activate sql
5 ## 启动命令
6 ## nohup python admin_server.py >> admin_server.log &
7 python admin_server.py
8
```

1.6 后台系统前端服务启动

代码块

```
1 ## 进入项目
2 cd /home/ubuntu/chat-to-db/frontend/admin
3 ## 切入虚拟环境
4 conda activate sql
```

```
5  ## 启动命令
6  PORT=8089 HOST=0.0.0.0 npm run dev
```

1.7 前台系统后端服务启动

代码块

```
1  ## 进入命令
2  cd /home/ubuntu/chat-to-db/backend
3  ## 切入虚拟环境
4  conda activate sql
5  ## 启动命令 nohup python chat_server.py >> chat_server.log &
6  python chat_server.py
```

1.8 前台系统前端服务启动

代码块

```
1  ## 进入项目
2  cd /home/ubuntu/chat-to-db/frontend/chat
3  ## 切入虚拟环境
4  conda activate sql
5  ## 启动命令
6  ## HOST=0.0.0.0 PORT=3000 npm start
7  PORT=3000 HOST=0.0.0.0 npx next dev
```

4. 智能问数后台管理系统设计

4.1 数据库设计

初始化数据库

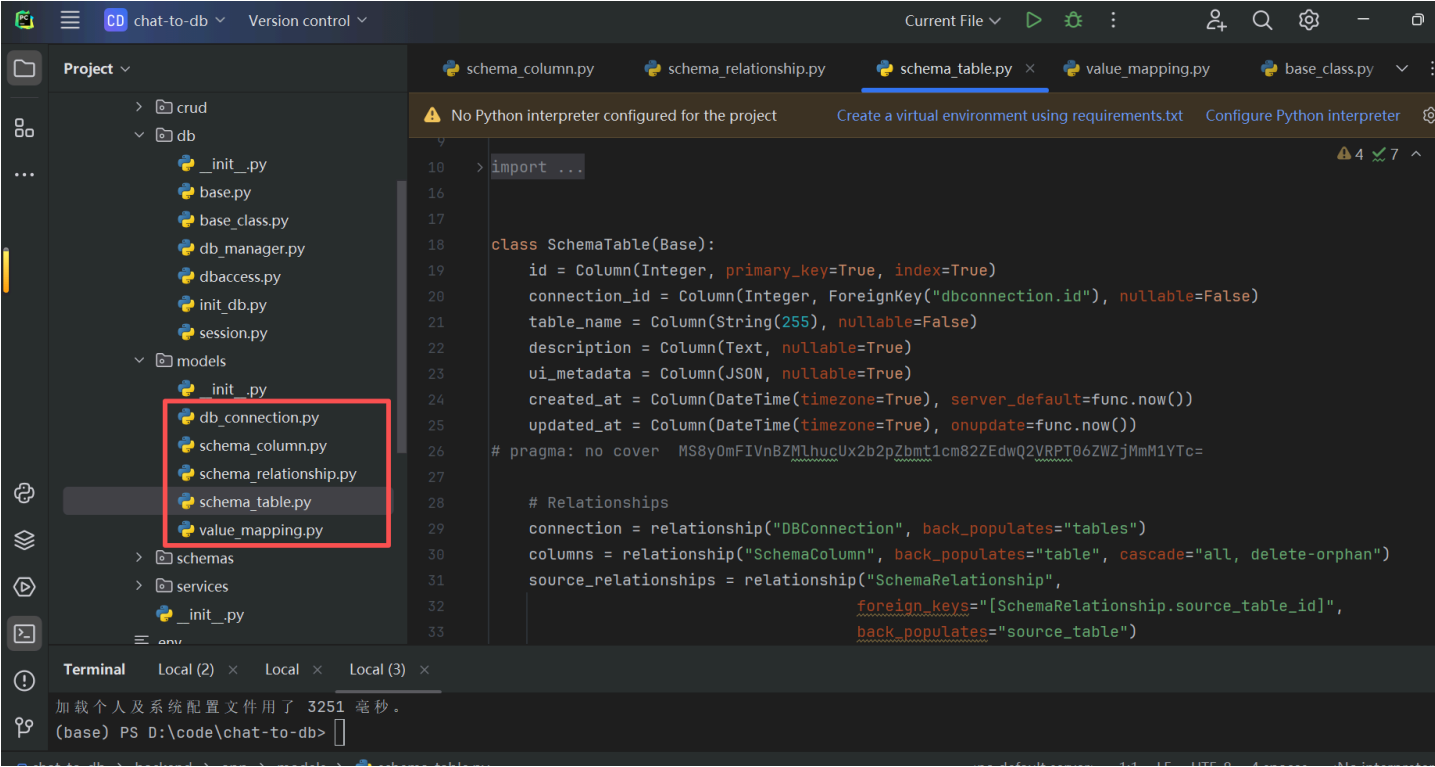
进入到db目录，执行如下语句

代码块

```
1  python init_db.py
```

数据库初始化完成！

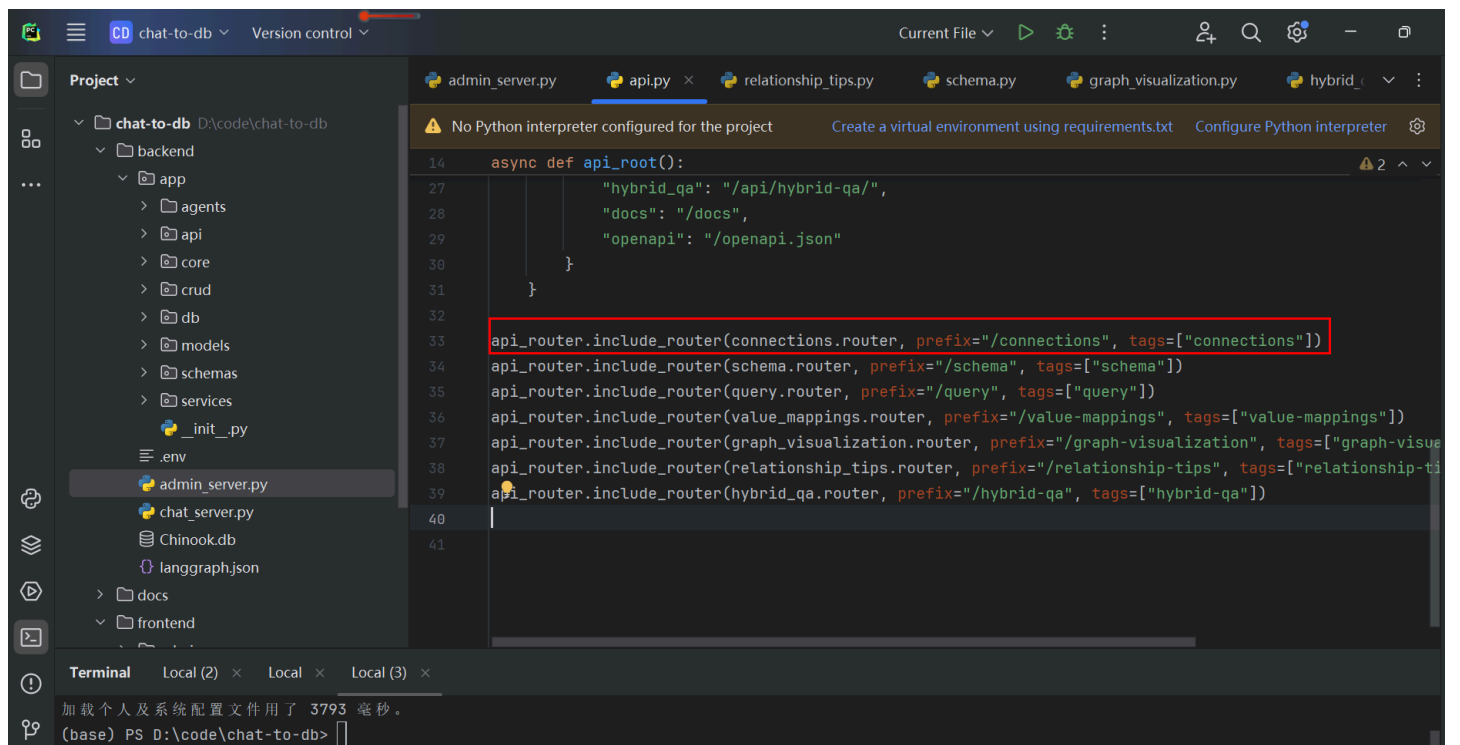
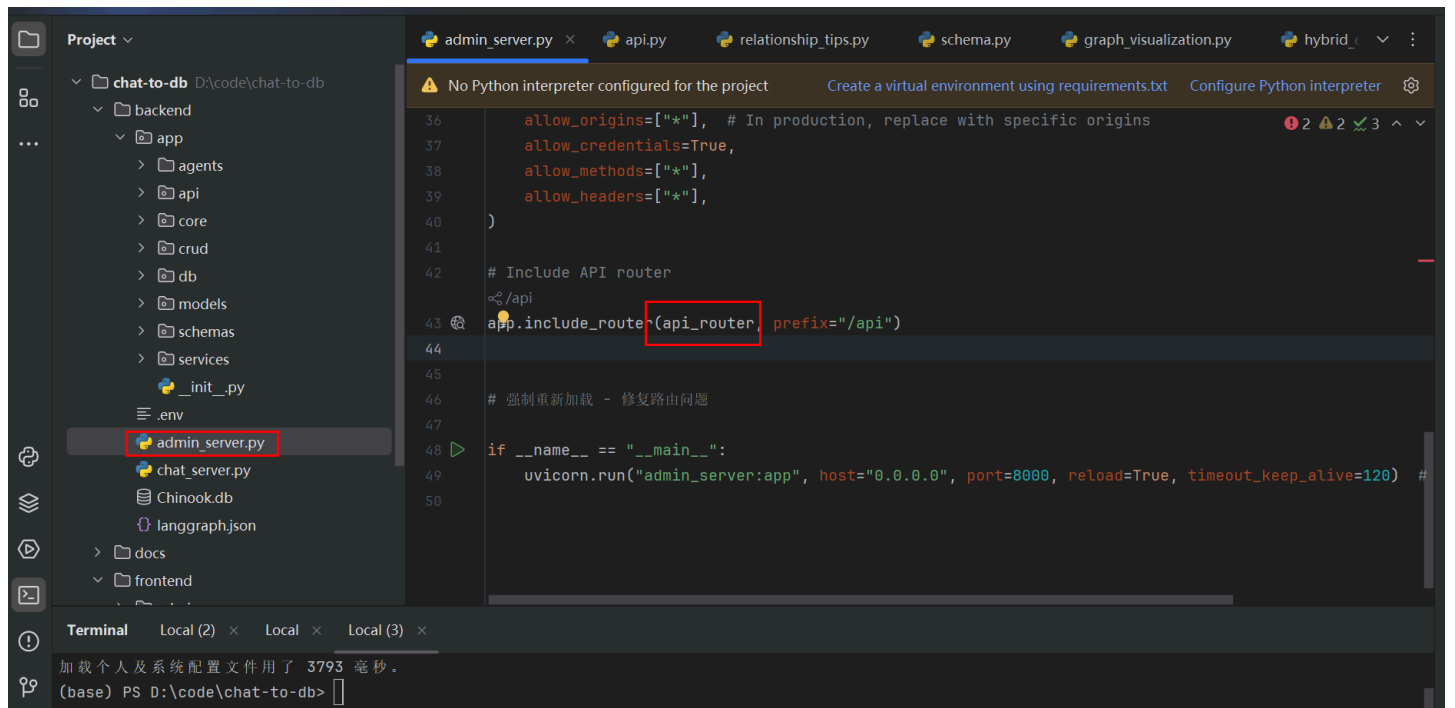
后台数据库的表有如下5表

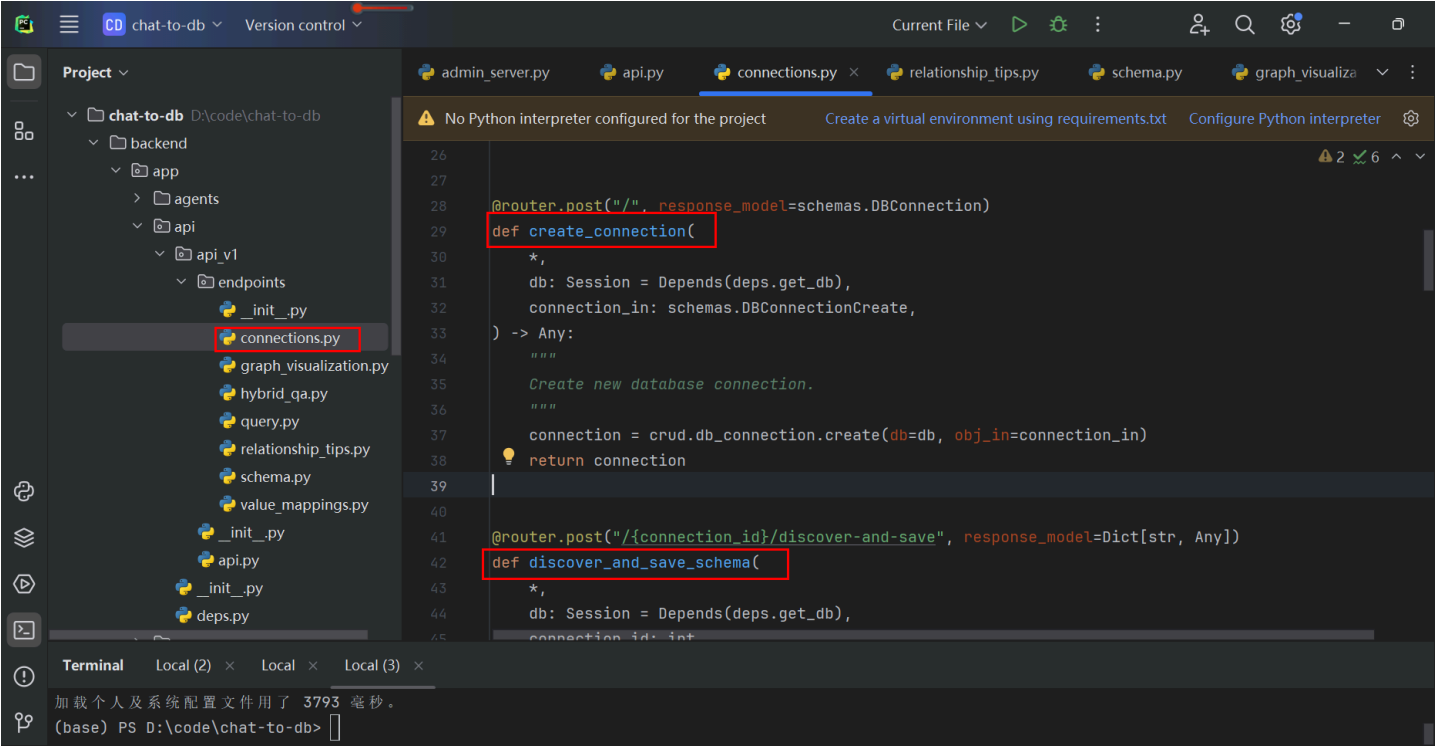


db_connection	被连接的数据库的信息
schema_column	字段信息
schema_relationship	表之间的关系
schema_table	表信息
value_mappping	词语映射管理

4.2 连接管理

核心代码

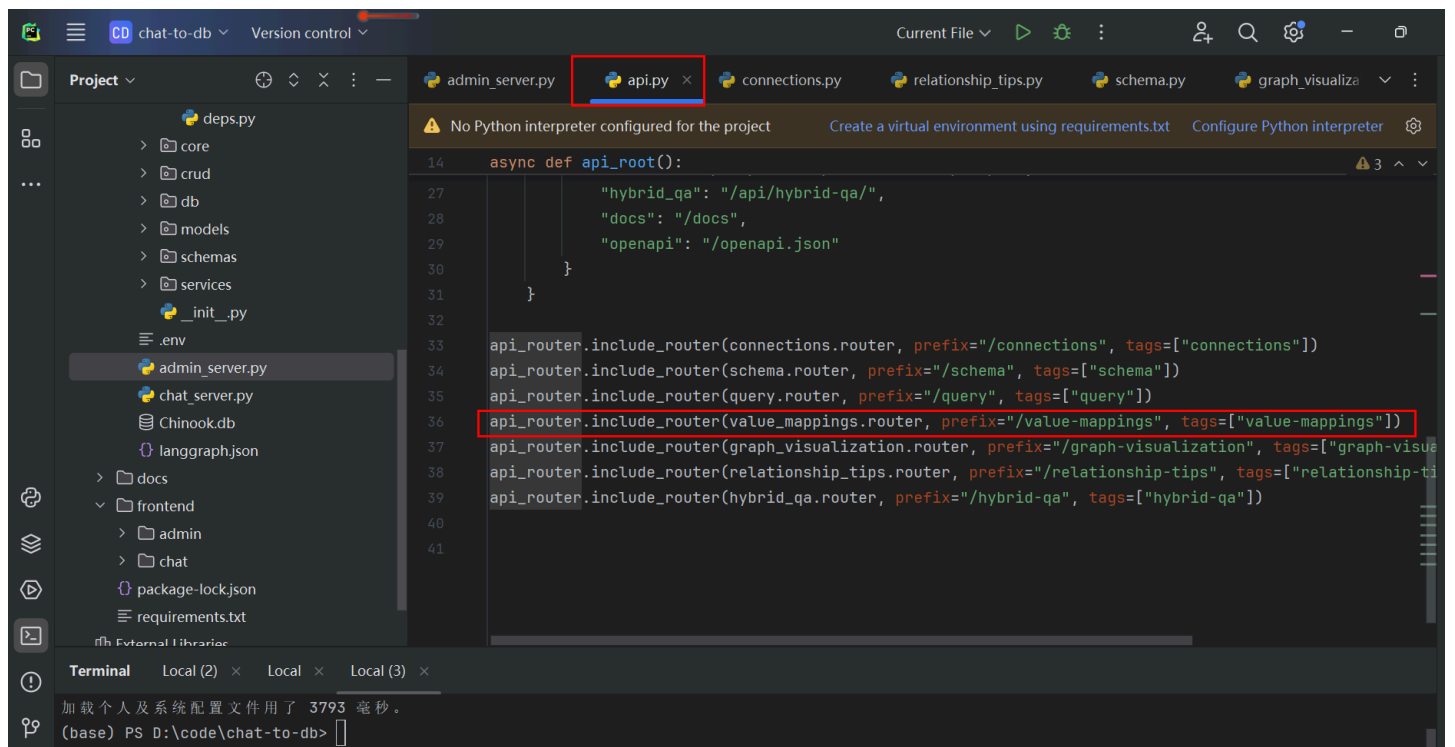
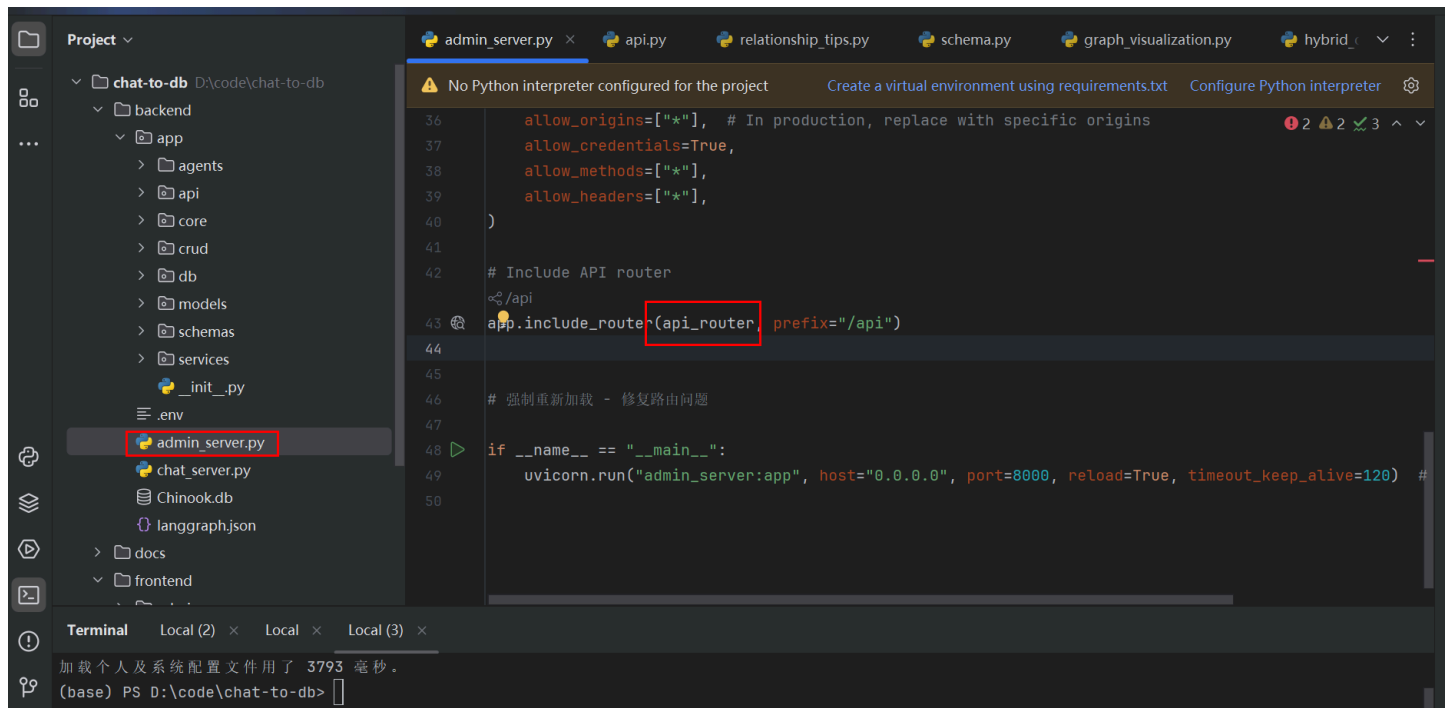


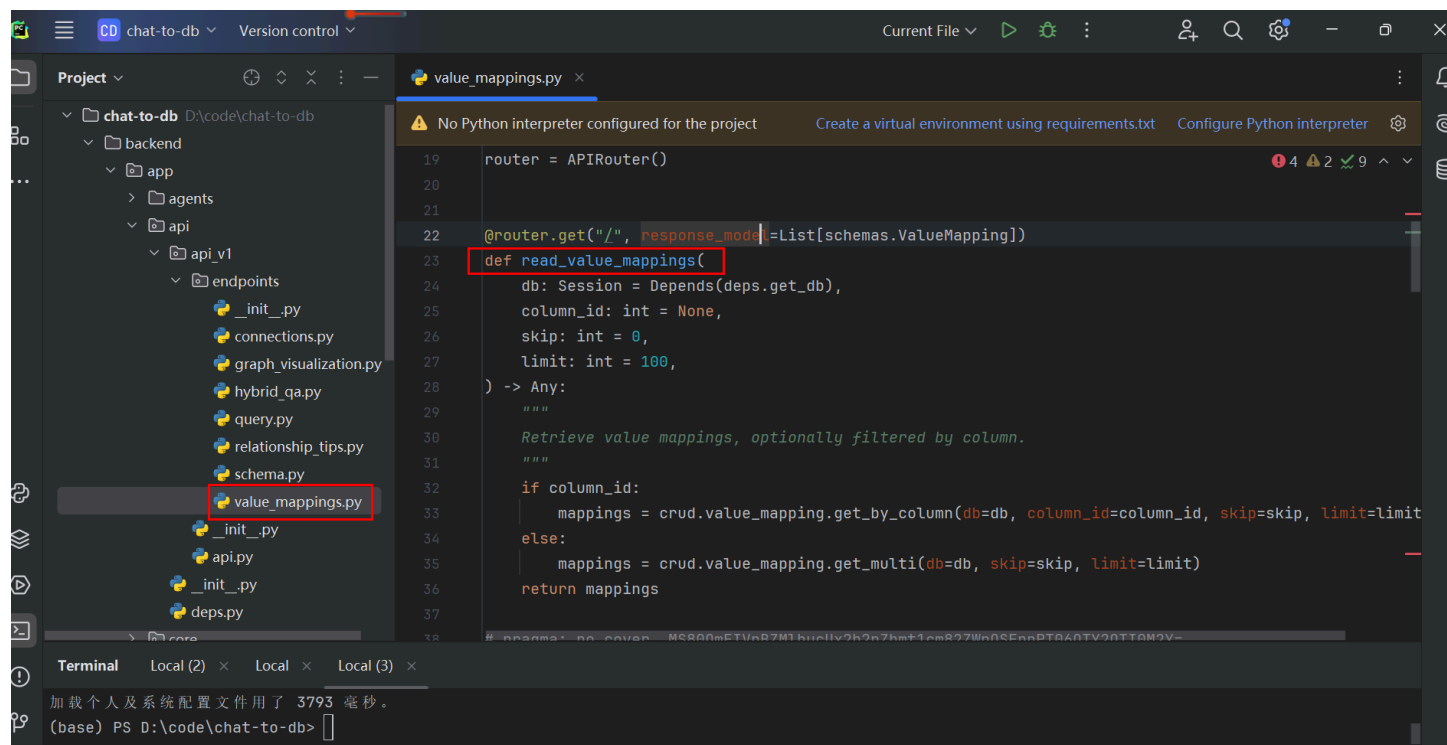


接口路径	请求方法	接口功能	核心逻辑
/	GET	获取所有数据库连接	分页查询数据库中所有的数据库连接配置信息
/	POST	创建数据库连接	新增一条数据库连接配置信息并保存到数据库
/ {connection_id} / discover-and-save	POST	发现并保存数据库结构	1. 校验连接是否存在2. 测试数据库连通性3. 自动解析库表结构4. 保存表结构和关联关系到数据库
/ {connection_id}	GET	获取单个数据库连接	根据 ID 查询指定的数据库连接配置详情
/ {connection_id}	PUT	更新数据库连接	根据 ID 修改指定的数据库连接配置信息
/ {connection_id}	DELETE	删除数据库连接	根据 ID 删除指定的数据库连接配置信息
/ {connection_id} / test	POST	测试数据库连接	校验指定 ID 的数据库连接是否能正常连通

4.3 数值映射

核心代码

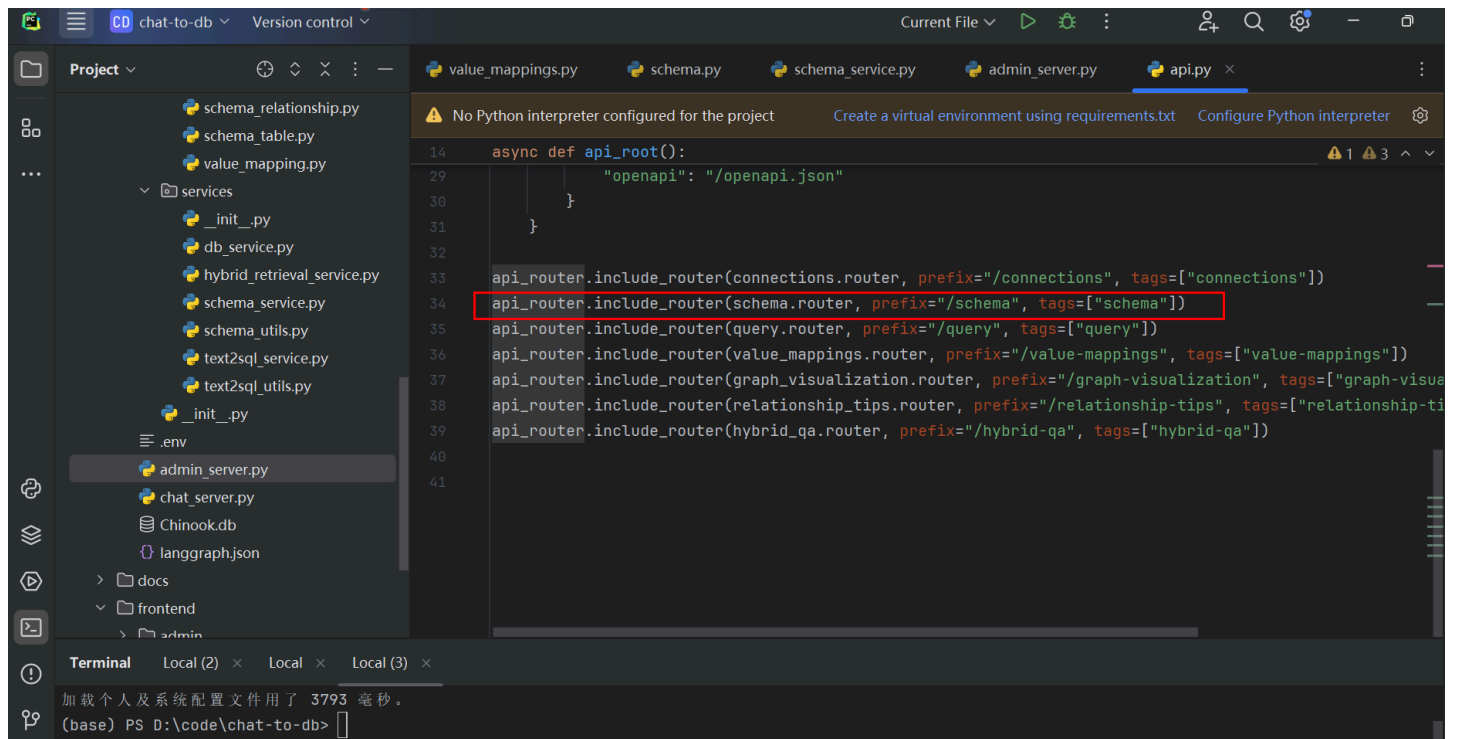
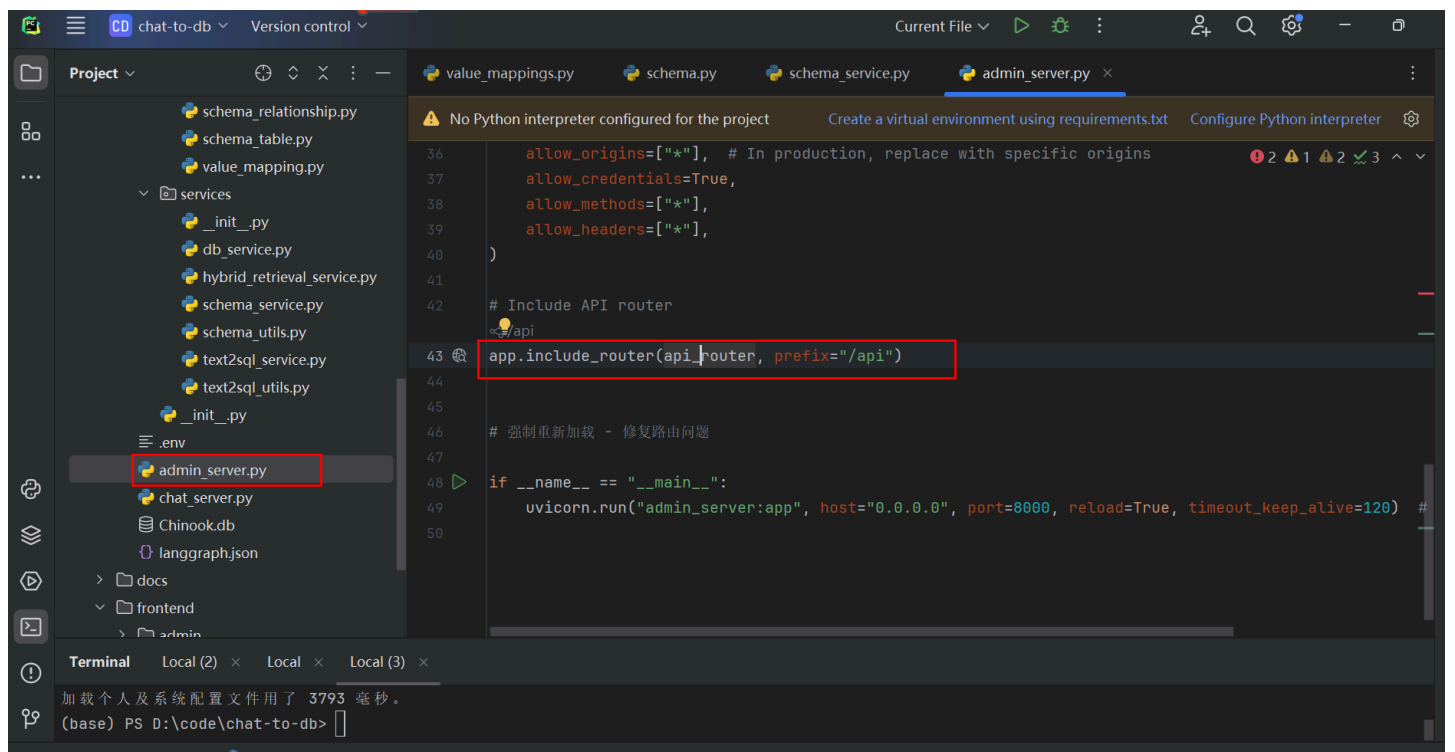


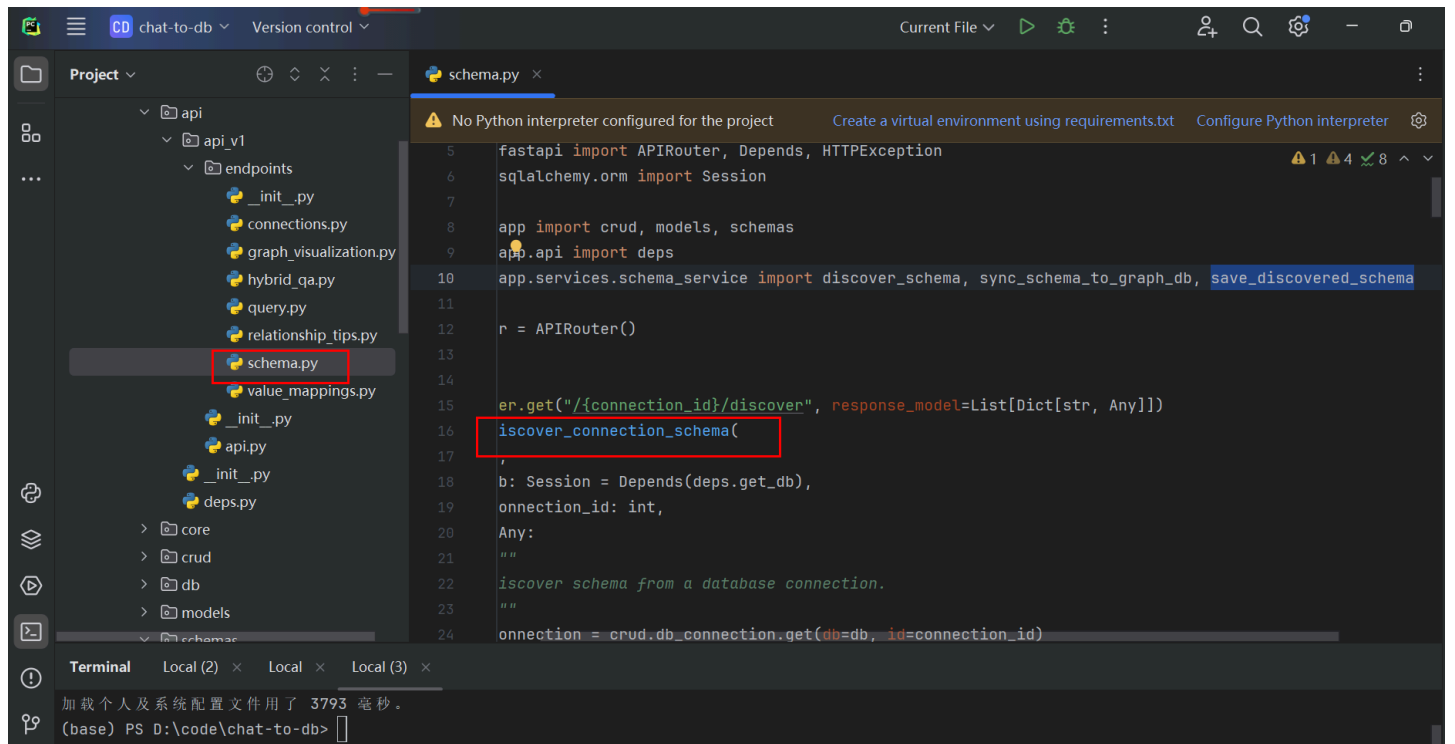


请求方式	接口路径	接口功能	核心参数	关键校验 / 逻辑
GET	/	获取值映射列表 (支持分页、按 列 ID 筛选)	column_id(可 选)、skip(分页 起始)、limit(分 页数量)	有column_id则按列筛选，无则查 询全部
POST	/	创建新的值映射	mapping_in(创 建参数)	1. 校验列是否存在2. 校验术语是否 重复3. 无重复则创建
GET	/ {mapping_id}	根据 ID 查询单 个值映射	mapping_id(路 径参数)	校验 ID 对应映射是否存在，不存在 则 404
PUT	/ {mapping_id}	根据 ID 更新值 映射	mapping_id、 mapping_in(更 新参数)	1. 校验映射是否存在2. 修改术语时 校验重复3. 校验通过则更新
DELETE	/ {mapping_id}	根据 ID 删除值 映射	mapping_id(路 径参数)	校验映射是否存在，存在则执行删 除

4.4 语义治理

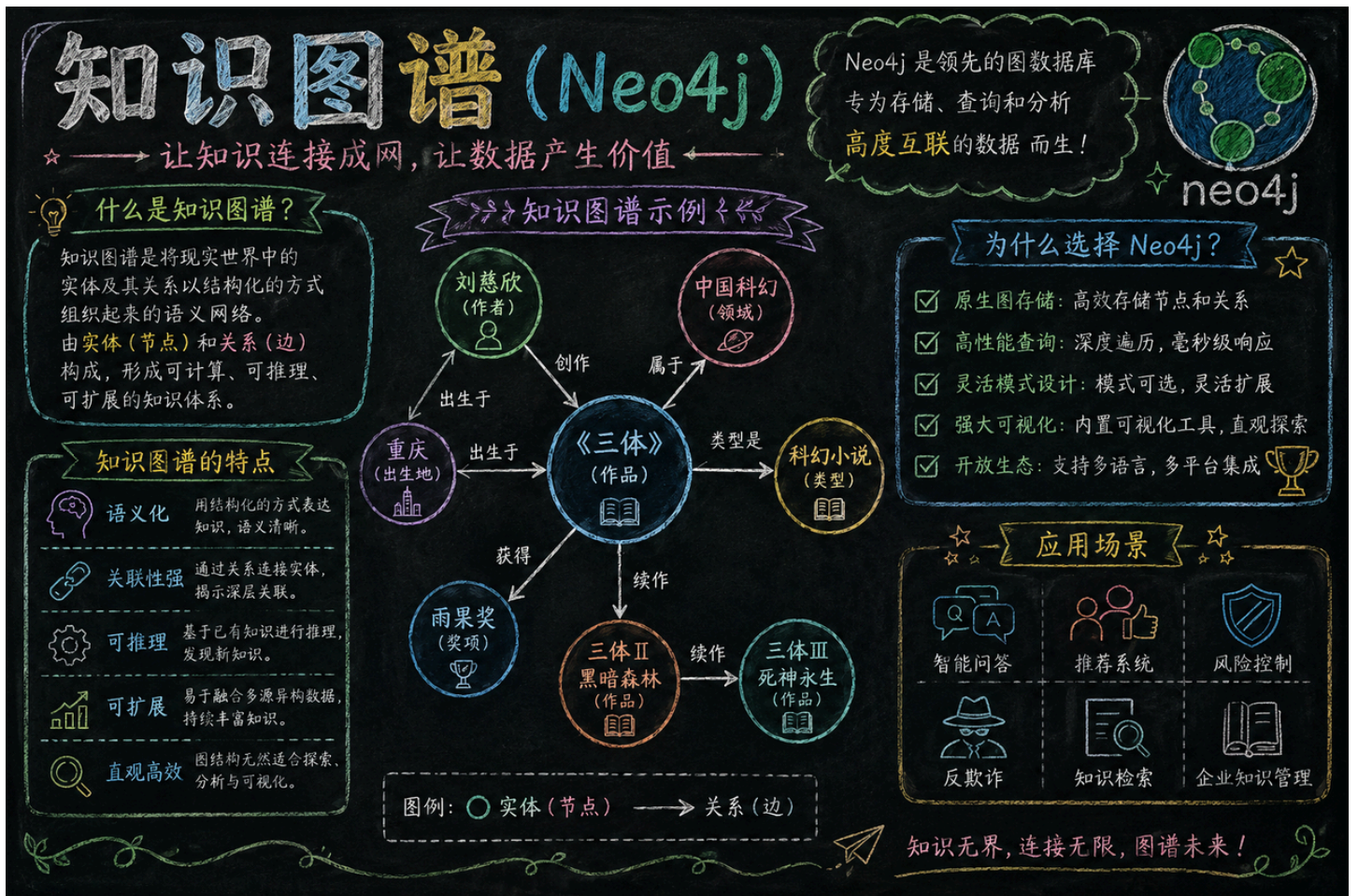
核心代码





接口路径	请求方法	接口函数	核心作用	详细功能描述（严格匹配代码）
/ {connection_id }/discover	GET	discover_conn ection_schema	实时探测数据库 结构	从目标数据源实时获取表、列结 构；匹配本地已存在的表 / 列 ID 并返回； 只查询、不修改本地 数据
/ {connection_id }/metadata	GET	get_schema_m etadata	获取本地完整元 数据	查询 MySQL 中维护的 表、列、 关联关系 ；自动组装详细的关 系信息（表名 / 列名）；返回标 准化 Pydantic 格式
/ {connection_id }/publish	POST	publish_schem a	发布并全量同步	接收前端编辑后的 Schema 数 据； 新增 / 更新 / 删除MySQL 中 的表、列、关系；同步到 Neo4j 图数据库
/ {connection_id }/saved	GET	get_saved_sch ema	获取带 UI 配置 的保存数据	查询本地已保存的表、列、关 系；返回包含ui_metadata的简 易格式；专供前端渲染 / 编辑
/ {connection_id }/sync-to-neo4j	POST	sync_to_neo4j	手动同步 Neo4j	仅执行：将 MySQL 的 Schema 数据 同步到 Neo4j ；不修改 MySQL，仅做图库同步
/ {connection_id }/discover-and- sync	POST	discover_and_s ync	探测并保存到 MySQL	仅做两件事 ：1. 从数据源探测 Schema 结构；2. 调用 save_discovered_schema保存到 MySQL
/tables/ {table_id}	PUT	update_table	单独更新表信息	修改指定表的描述、UI 元数据
/columns/ {column_id}	PUT	update_colum n	单独更新列信息	修改指定列的描述、主键 / 外键 标识

4.5 知识图谱设计



核心代码

```
410
411
412 @router.post("/{connection_id}/sync-to-neo4j", response_model=Dict[str, Any])
413 def sync_to_neo4j(
414     *,
415     db: Session = Depends(deps.get_db),
416     connection_id: int,
417 ) -> Any:
418     """
419     Manually sync schema metadata to Neo4j graph database.
420     """
421     connection = crud.db_connection.get(db=db, id=connection_id)
422     if not connection:
423         raise HTTPException(status_code=404, detail="Connection not found")
424
425     try:
426         # Sync to Graph DB
427         result = sync_schema_to_graph_db(connection_id)
428         if result:
429             return {"status": "success", "message": "Schema synced to Neo4j successfully"}
```

代码块

```
1 def sync_schema_to_graph_db(connection_id: int):
2     """
3     Sync schema metadata to Neo4j graph database.
```

```

4         """
5         try:
6             print(f"Starting sync to Neo4j for connection_id: {connection_id}")
7             # Connect to Neo4j
8             print(f"Connecting to Neo4j at {settings.NEO4J_URI} with user
{settings.NEO4J_USER}")
9             driver = GraphDatabase.driver(
10                 settings.NEO4J_URI,
11                 auth=(settings.NEO4J_USER, settings.NEO4J_PASSWORD)
12             )
13
14             with driver.session() as session:
15                 # Clear existing schema for this connection
16                 print(f"Clearing existing schema for connection_id:
{connection_id}")
17                 session.run(
18                     "MATCH (n {connection_id: $connection_id}) DETACH DELETE n",
19                     connection_id=connection_id
20                 )
21
22                 # Get all tables for this connection from MySQL
23                 from sqlalchemy.orm import Session
24                 from app.db.session import SessionLocal
25
26                 db = SessionLocal()
27                 try:
28                     print(f"Fetching tables for connection_id: {connection_id}")
29                     tables = crud.schema_table.get_by_connection(db=db,
connection_id=connection_id)
30                     print(f"Found {len(tables)} tables for connection_id:
{connection_id}")
31
32                     if len(tables) == 0:
33                         print(f"Warning: No tables found for connection_id:
{connection_id}")
34                         return False
35
36                 # Create Table nodes
37                 for table in tables:
38                     print(f"Creating Table node for table: {table.table_name}
(id: {table.id})")
39                     session.run(
40                         """
41                         CREATE (t:Table {
42                             id: $id,
43                             connection_id: $connection_id,
44                             name: $name,

```

```

45         description: $description
46     })
47     """
48     id=table.id,
49     connection_id=connection_id,
50     name=table.table_name,
51     description=table.description or ""
52 )
53
54     # Get columns for this table
55     columns = crud.schema_column.get_by_table(db=db,
table_id=table.id)
56     print(f"Found {len(columns)} columns for table:
{table.table_name}")
57
58     # Create Column nodes and HAS_COLUMN relationships
59     for column in columns:
60         print(f"Creating Column node for column:
{column.column_name} (id: {column.id})")
61         session.run(
62             """
63             CREATE (c:Column {
64                 id: $id,
65                 name: $name,
66                 type: $type,
67                 description: $description,
68                 is_pk: $is_pk,
69                 is_fk: $is_fk,
70                 connection_id: $connection_id
71             })
72             WITH c
73             MATCH (t:Table {id: $table_id})
74             CREATE (t)-[:HAS_COLUMN]->(c)
75             """
76             id=column.id,
77             name=column.column_name,
78             type=column.data_type,
79             description=column.description or "",
80             is_pk=column.is_primary_key,
81             is_fk=column.is_foreign_key,
82             connection_id=connection_id,
83             table_id=table.id
84         )
85
86     # Create FOREIGN_KEY relationships
87     relationships =
crud.schema_relationship.get_by_connection(db=db, connection_id=connection_id)

```

```

88         print(f"Found {len(relationships)} relationships for
connection_id: {connection_id}")
89
90         for rel in relationships:
91             print(f"Creating REFERENCES relationship from column id:
{rel.source_column_id} to column id: {rel.target_column_id}")
92             session.run(
93                 """
94                 MATCH (source:Column {id: $source_column_id})
95                 MATCH (target:Column {id: $target_column_id})
96                 CREATE (source)-[:REFERENCES {
97                     type: $relationship_type,
98                     description: $description,
99                     connection_id: $connection_id
100                 }]->(target)
101                 """,
102                 source_column_id=rel.source_column_id,
103                 target_column_id=rel.target_column_id,
104                 relationship_type=rel.relationship_type or "unknown",
105                 description=rel.description or "",
106                 connection_id=connection_id
107             )
108
109             print(f"Successfully synced schema to Neo4j for connection_id:
{connection_id}")
110         finally:
111             db.close()
112
113     driver.close()
114     return True
115 except Exception as e:
116     import traceback
117     error_trace = traceback.format_exc()
118     print(f"Graph DB sync failed: {str(e)}\n{error_trace}")
119     raise Exception(f"Graph DB sync failed: {str(e)}")

```

1. 建立 Neo4j 连接：根据配置的连接信息（URI、用户名、密码），初始化并建立与 Neo4j 图数据库的驱动连接。
2. 清空旧数据：为了防止数据重复或冲突，首先通过 Cypher 语句（`MATCH...DETACH DELETE`）删除图数据库中所有带有当前 `connection_id` 标签的旧节点和关系。
3. 获取并遍历表信息：从本地 MySQL 数据库中拉取该连接下的所有表（Table）元数据，并逐一遍历。
4. 创建表与字段的图节点：

- 为每个表创建一个 `Table` 节点；
 - 获取该表下的所有字段（Column），为每个字段创建一个 `Column` 节点，并建立从表指向字段的 `HAS_COLUMN` 关系。
5. 建立字段间的关联关系：从 MySQL 中获取表与表之间的外键或逻辑关联关系（Relationship），在图数据库中通过 `MATCH` 找到对应的源字段和目标字段节点，并创建 `REFERENCES` 关系。
 6. 资源释放与异常处理：同步完成后，关闭本地数据库会话和 Neo4j 驱动连接；如果在过程中出现任何异常，会捕获并打印详细的堆栈错误信息。

模拟的 MySQL 元数据（输入数据）

假设 MySQL 中有两张表：`users`（用户表）和 `orders`（订单表）

假设当前 `connection_id = 1001`，你的 MySQL 元数据表中存了以下信息：

1. 表信息 (schema_table)

- 表A: `id=10`, `table_name="users"`, `description="用户信息表"`
- 表B: `id=20`, `table_name="orders"`, `description="订单记录表"`

2. 字段信息 (schema_column)

• 属于 `users` 表的字段：

- `id=101`, `column_name="user_id"`, `data_type="INT"`, `is_primary_key=True`
- `id=102`, `column_name="username"`, `data_type="VARCHAR"`, `is_primary_key=False`

• 属于 `orders` 表的字段：

- `id=201`, `column_name="order_id"`, `data_type="INT"`, `is_primary_key=True`
- `id=202`, `column_name="buyer_id"`, `data_type="INT"`, `is_foreign_key=True` (注：该外键关联到 `users` 表的 `user_id`)

3. 关联关系信息 (schema_relationship)

- 关系: `source_column_id=202` (`orders.buyer_id`) 指向 `target_column_id=101` (`users.user_id`)

代码执行后的 Neo4j 图结构（输出结果）

当代码跑完后，Neo4j 图数据库中会生成以下节点和关系：

4. 清空阶段

- 所有带有 `connection_id: 1001` 属性的旧节点和关系会被全部删除。

5. 节点创建阶段 代码会创建 4 个字段节点和 2 个表节点，例如：

• **Table 节点:**

- `(:Table {id: 10, name: "users", connection_id: 1001})`
- `(:Table {id: 20, name: "orders", connection_id: 1001})`

• **Column 节点:**

- `(:Column {id: 101, name: "user_id", type: "INT", is_pk: True, connection_id: 1001})`
- `(:Column {id: 202, name: "buyer_id", type: "INT", is_fk: True, connection_id: 1001})`
- (以及其他两个字段节点)

6. 关系构建阶段

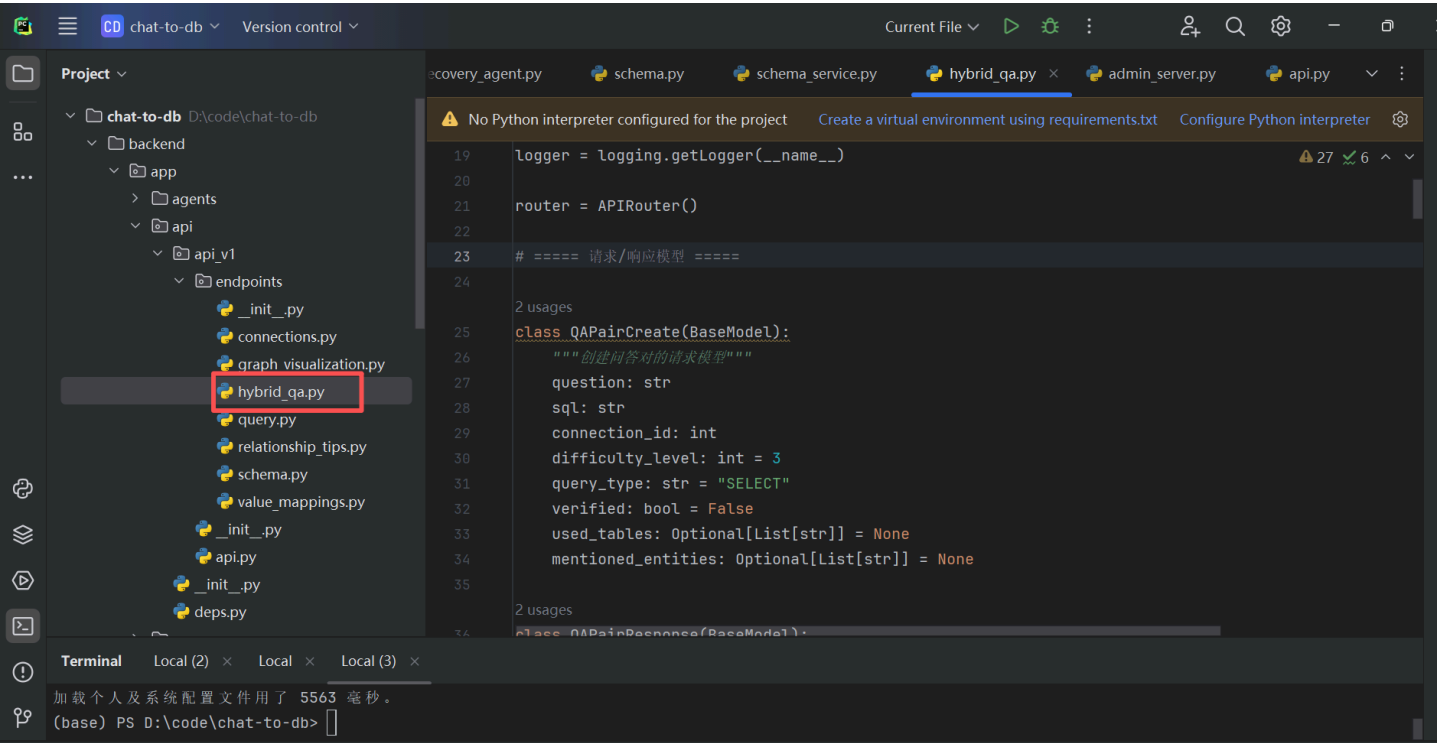
• **表与字段的包含关系 (HAS_COLUMN):**

- `users` 表节点会伸出箭头指向 `user_id` 和 `username` 字段节点。
- `orders` 表节点会伸出箭头指向 `order_id` 和 `buyer_id` 字段节点。

• **字段间的外键引用关系 (REFERENCES):**

- `buyer_id` 字段节点会伸出一个带有 `REFERENCES` 标签的箭头，直接指向 `user_id` 字段节点。

4.6 智能训练



接口路径	HTTP方法	功能说明	核心逻辑	输入参数	输出响应
------	--------	------	------	------	------

/qa-pairs/	POST	创建问答对	自动提取表名和实体，生成问答对并存储到混合检索引擎	QAPairCreate : question, sql, connection_id, difficulty_level, query_type, verified, used_tables, mentioned_entities	{status, qa_id, message}
/qa-pairs/search	POST	搜索相似问答对	执行混合检索（语义+结构+模式+质量），返回相似问答对列表	SearchRequest : question, connection_id, schema_context, top_k	List[SimilarQAPairResponse]（含 semantic_score, structural_score 等评分）
/qa-pairs/stats	GET	获取问答对统计信息	返回问答对总数、验证数量、查询类型分布、难度分布、平均成功率	connection_id（可选 Query 参数）	{total_qa_pairs, verified_qa_pairs, query_types, difficulty_distribution, average_success_rate}
/qa-pairs/feedback	POST	提交用户反馈	记录用户对问答对的满意度反馈，用于更新成功率	FeedbackRequest : qa_id, user_satisfaction, feedback_text	{status, message}
/qa-pairs/health	GET	健康检查	检查混合检索引擎及各依赖服务（Milvus/Neo4j/Vector）状态	无	{status, services, timestamp}
/qa-pairs/batch-create	POST	批量创建问答对	循环处理问答对列表，自动提取元数据并存储，	List[QAPairCreate]	{status, created_count,

			支持部分失败容错		<code>failed_count</code> <code>, errors}</code>
<code>/qa-pairs/{qa_id}</code>	DELETE	删除问答对	从Neo4j和Milvus中删除指定问答对数据（待实现）	<code>qa_id</code> （路径参数）	<code>{status, message}</code>
<code>/qa-pairs/export</code>	GET	导出问答对数据	按指定格式导出问答对数据（开发中）	<code>connection_id</code> （可选）， <code>format</code> （json/csv）	<code>{status, message, format, connection_id}</code>

检索维度	核心数据源	匹配逻辑	适用场景	权重动态调整
语义检索 (Semantic)	Milvus向量库	问题文本的向量相似度（余弦距离）	用户问法不同但意图相同	语义分 ≥ 0.9 时权重升至80%
结构检索 (Structural)	Neo4j图数据库	Schema中表名/字段的重合度	查询涉及相同数据表	语义分 ≤ 0.5 时权重升至35%
模式检索 (Pattern)	Neo4j图数据库	查询类型 (SELECT/JOIN/AGG等) + 难度级别	相同SQL模式的不同业务场景	语义分 ≤ 0.5 时权重升至20%

已存储的问答对示例：

QA ID	问题	SQL	涉及表	查询类型	难度	验证状态	成功率
qa_001	"查询最近30天下单的用户"	<code>SELECT u.name FROM users u JOIN orders o ON u.id=o.user_id WHERE o.created_at ></code>	users, orders	JOIN	3		0.95

		DATE_SUB(NOW(),30 DAY)					
qa_002	"统计每个用户的订单总数"	SELECT user_id, COUNT(*) FROM orders GROUP BY user_id	orders	GROUP_BY	2	✓	0.88
qa_003	"查找购买了iPhone的用户"	SELECT u.name FROM users u JOIN orders o ON u.id=o.user_id JOIN products p ON o.product_id=p.id WHERE p.name='iPhone'	users, orders, products	JOIN	4	✗	0.60
qa_004	"所有用户列表"	SELECT * FROM users	users	SELECT	1	✓	0.92

案例1：语义检索主导（用户问法高度相似）

用户提问：

"找出最近一个月有购买行为的客户"

检索维度	执行过程	得分计算
语义检索	向量编码后与qa_001的"最近30天下单的用户"余弦相似度	0.92 (高度相似)
结构检索	Schema涉及 users + orders ，与	1.0 (2/2表重合)

	qa_001完全重合	
模式检索	识别为JOIN类型，难度3，匹配qa_001	0.95 (模式+难度均匹配)

动态权重调整（语义≥0.9 → 语义优先）

代码块

```
1  final_score = 0.92×0.80 + 1.0×0.10 + 0.95×0.05 + quality×0.05 = 0.936 + 0.10 + 0.0475 + 0.0475 ≈ 0.93
```

案例2：结构检索主导（语义模糊但表结构明确）

用户提问： "把用户表和订单表连起来看看"

检索维度	执行过程	得分计算
语义检索	"连起来看看"语义模糊，与所有QA向量相似度均低	0.35 (低语义匹配)
结构检索	Schema仅涉及 users + orders ， qa_001/002/004均涉及	qa_001: 1.0 (2表全重合), qa_002: 0.5 (仅orders)
模式检索	识别为JOIN类型，匹配 qa_001/003	qa_001: 0.95, qa_003: 0.60

动态权重调整（语义<<0.5 → 结构模式优先）：

代码块

```
1  qa_001 final_score = 0.35×0.40 + 1.0×0.35 + 0.95×0.20 + 0.47×0.05 = 0.14 + 0.35 + 0.19 + 0.0235 ≈ 0.70
2  qa_003 final_score = 0.30×0.40 + 0.50×0.35 + 0.60×0.20 + 0.32×0.05 = 0.12 + 0.175 + 0.12 + 0.016 ≈ 0.43
```

结果排序： qa_001 排第一（结构重合度高，尽管语义模糊）

案例3：模式检索主导（相同SQL模式换业务场景）

用户提问： "统计每个部门的员工人数" （数据库表为 departments + employees ）

检索维度	执行过程	得分计算
语义检索	"部门""员工"与电商场景QA 语义差异大	0.25 (跨领域低匹配)
结构检索	Schema涉及 departments + employees，与电商表完全不重合	0.0 (无表重合)
模式检索	识别为GROUP_BY类型，难度2，匹配qa_002	0.90 (模式完全匹配)

动态权重调整（语义<=0.5）：

代码块

```
1  qa_002 final_score = 0.25×0.40 + 0.0×0.35 + 0.90×0.20 + 0.44×0.05 = 0.10 + 0 + 0.18 + 0.022 ≈ 0.30
```

结果排序： qa_002 排第一（虽表不同，但 GROUP BY + COUNT 模式完全匹配）

代码块

```
1  # 伪代码示意
2  if semantic_score >= 0.9:
3      权重 = [语义80%，结构10%，模式5%，质量5%] # 语义高度确信
4  elif semantic_score >= 0.7:
5      权重 = [语义70%，结构15%，模式10%，质量5%] # 语义较确信
6  elif semantic_score >= 0.5:
7      权重 = [语义50%，结构30%，模式15%，质量5%] # 基础权重
8  else:
9      权重 = [语义40%，结构35%，模式20%，质量5%] # 语义弱，依赖结构/模式
```

5. 智能问数前台系统设计

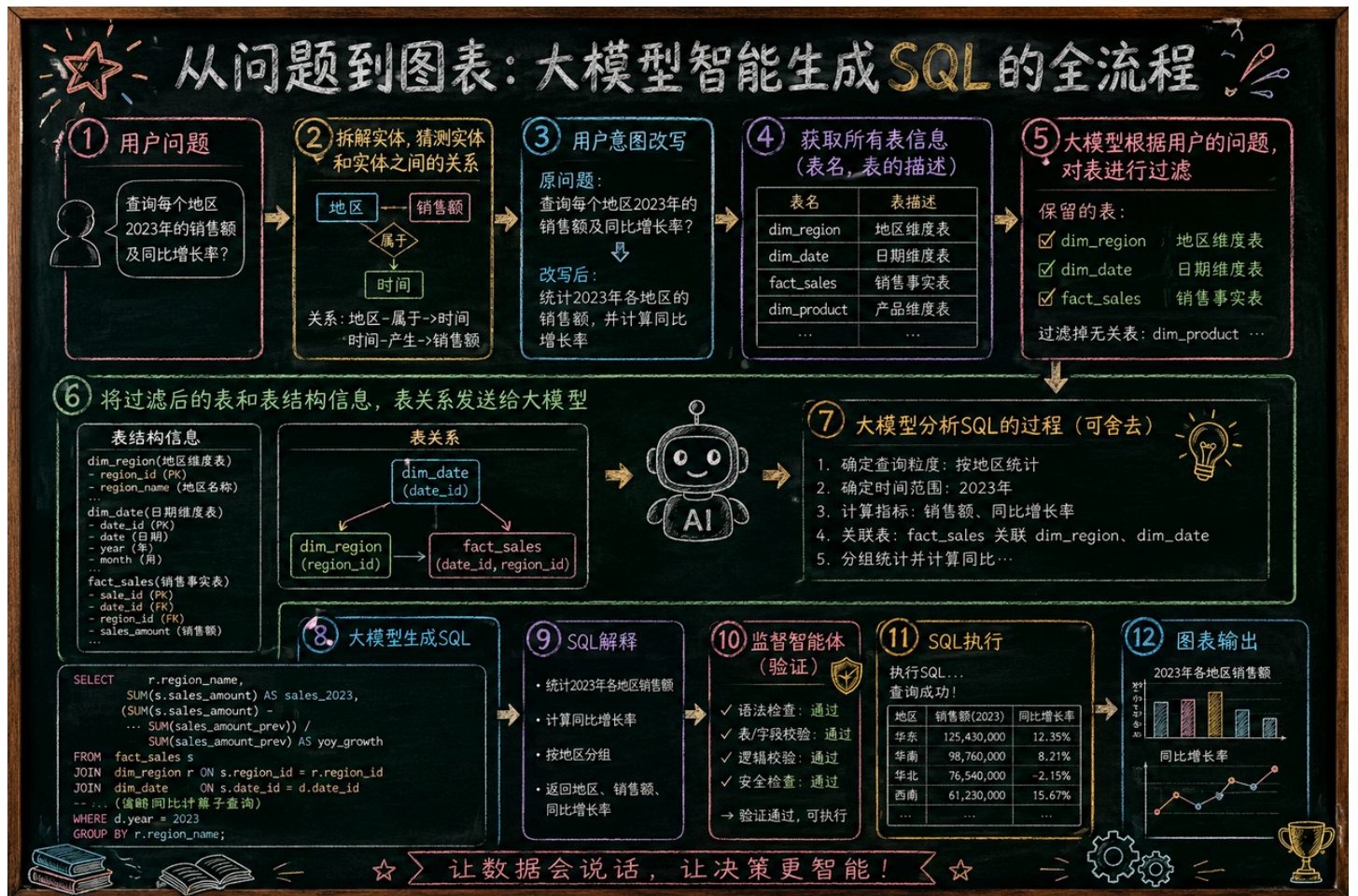
5.1 前台系统总体设计

所有的表的名字，表的描述，表的ID编号（当前库下面所有的表，1000表？2000表？），没有表的字段的

Token 大？不大！ 200个字段 -> 上下文的窗口/ token体积太大，影响大模型的注意力

1000张表 -> 给到大模型（能力强的模型）-> 3张表 -> Neo4j -> 1. 通过表去找到对应的列的信息。
2.
3 -> 5张表

1. 表的信息/词语映射/样本数据



5.2 LangGraph核心知识准备

LangChain rag - agent / Chain (xx |xx |xx |)

LangGraph Graph

LangChain (LangChain + LangGraph)

官网地址：<https://docs.langchain.com/oss/python/langgraph/overview>

Agent 创建：<https://docs.langchain.com/oss/python/langchain/agents>

结构化输出：<https://docs.langchain.com/oss/python/langchain/structured-output>

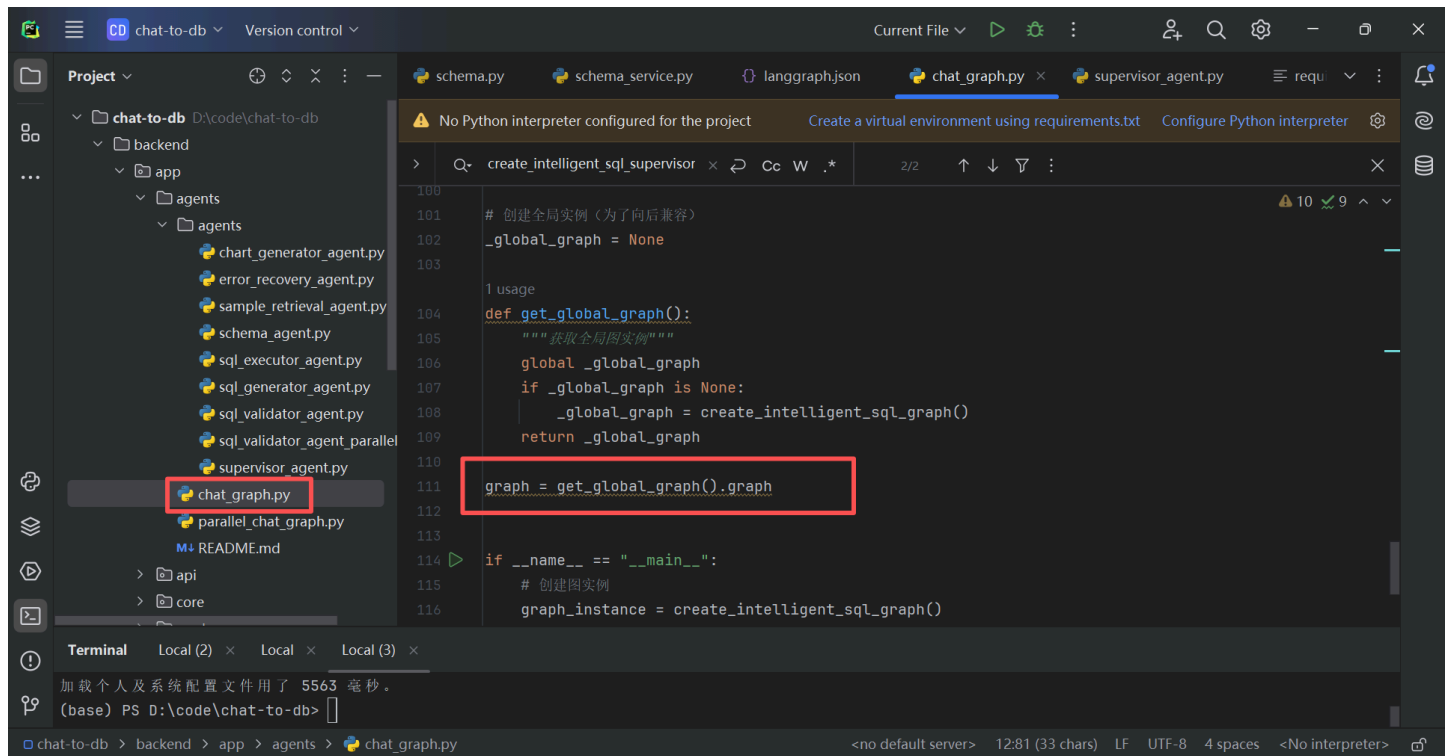
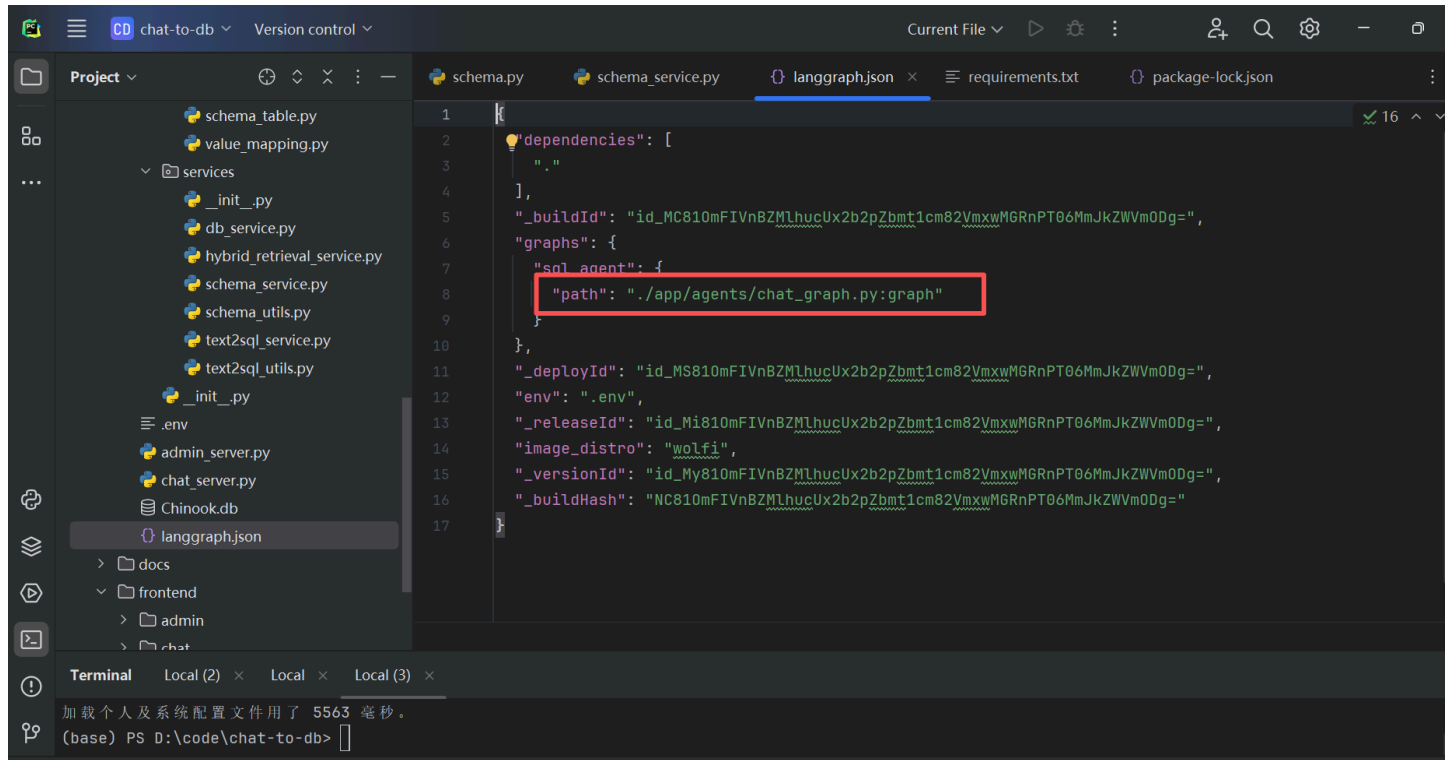
MCP 接入：<https://docs.langchain.com/oss/python/langchain/mcp>

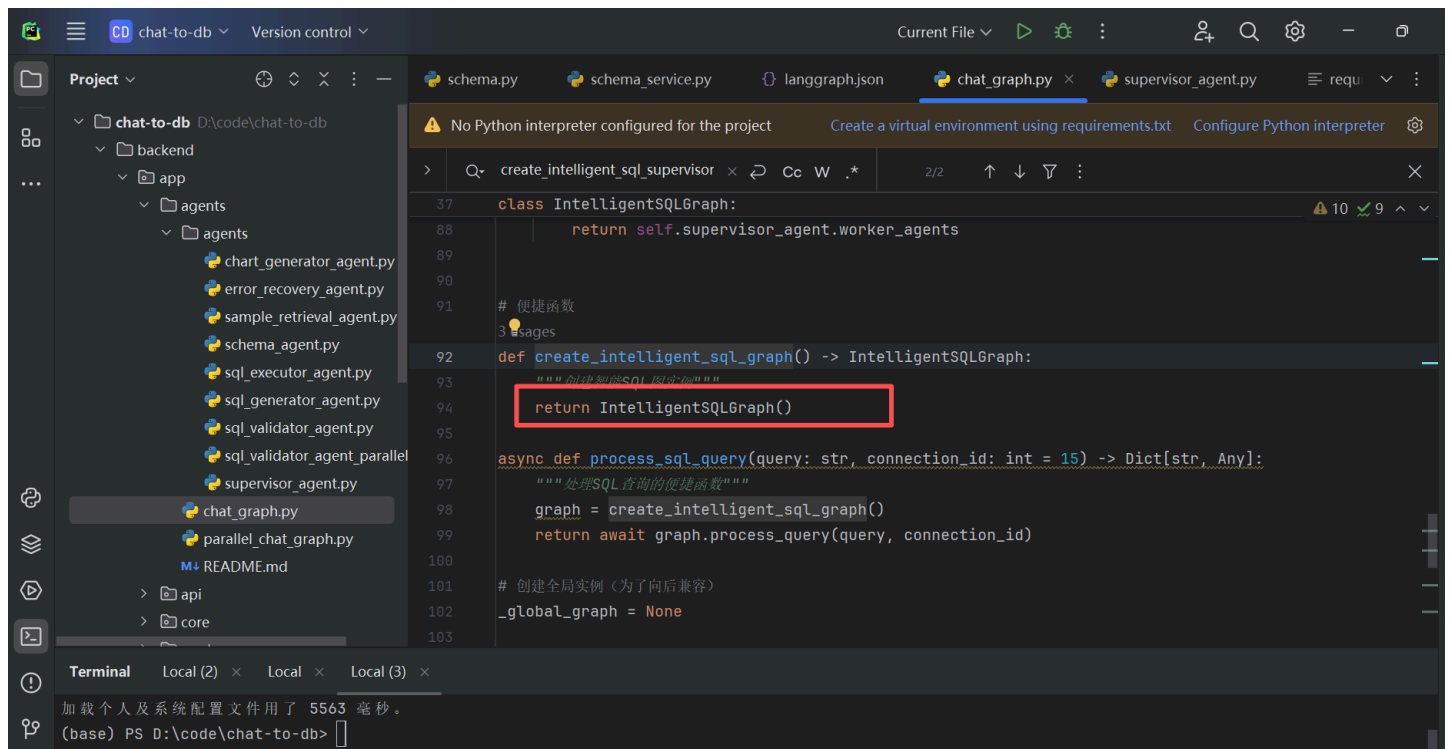
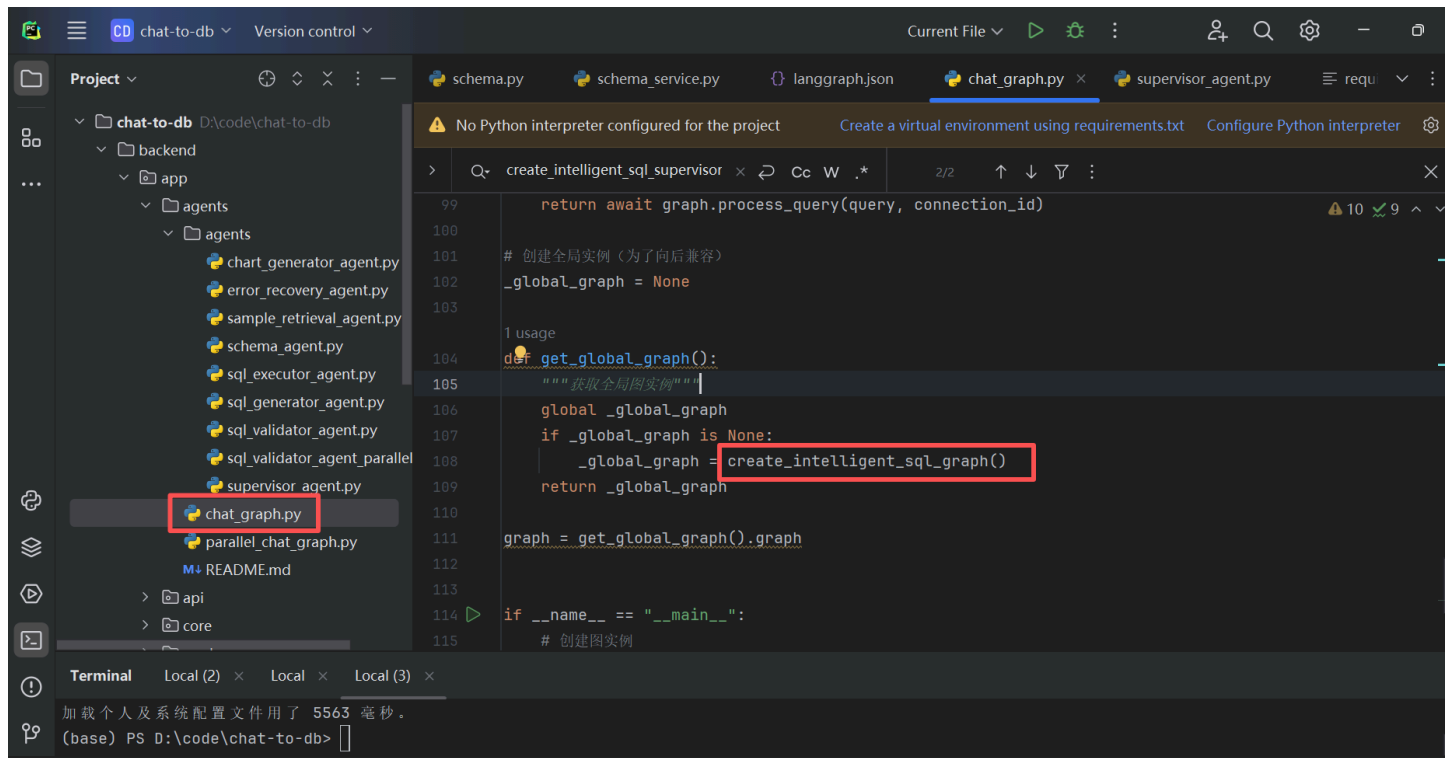
Supervisor Agent 设计: <https://docs.langchain.com/oss/python/langchain/multi-agent/subagents-personal-assistant>

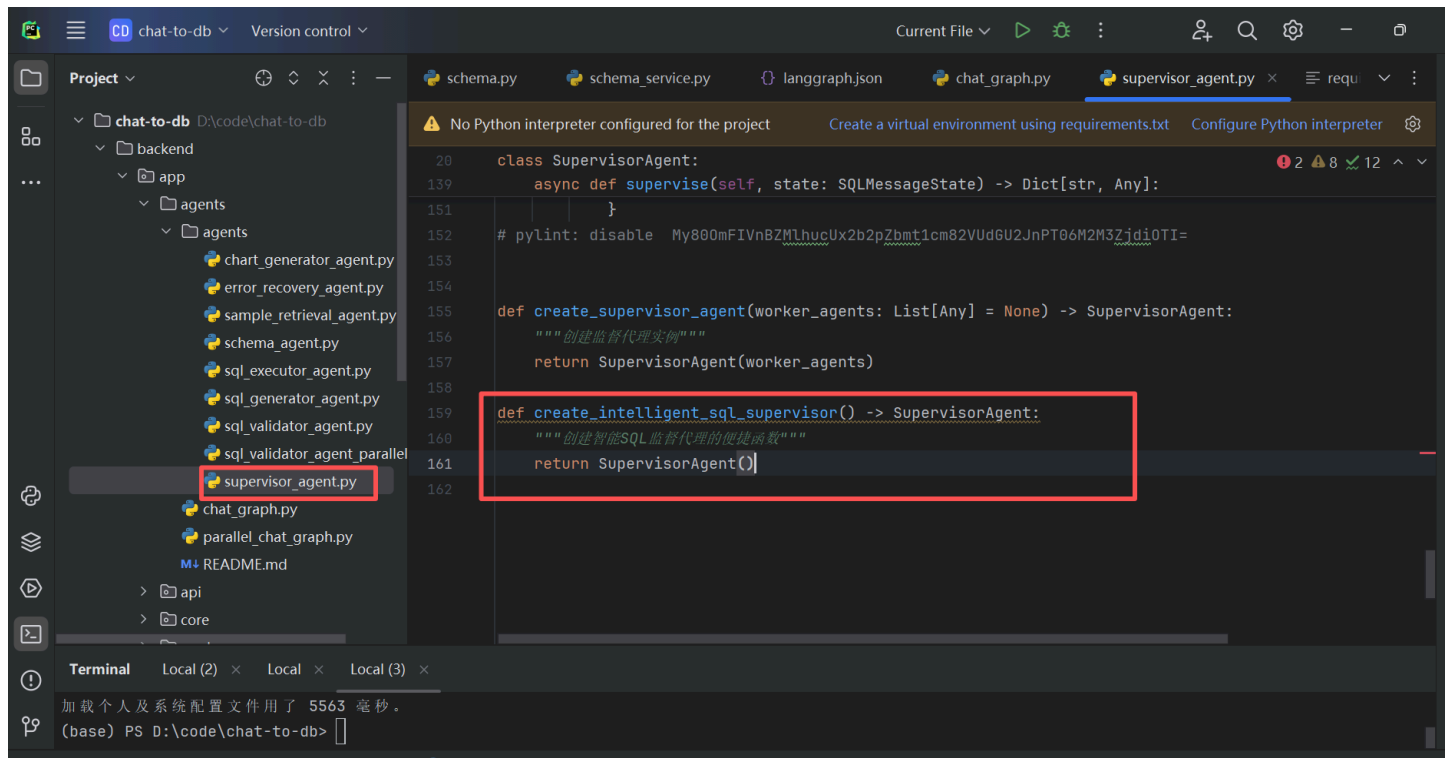
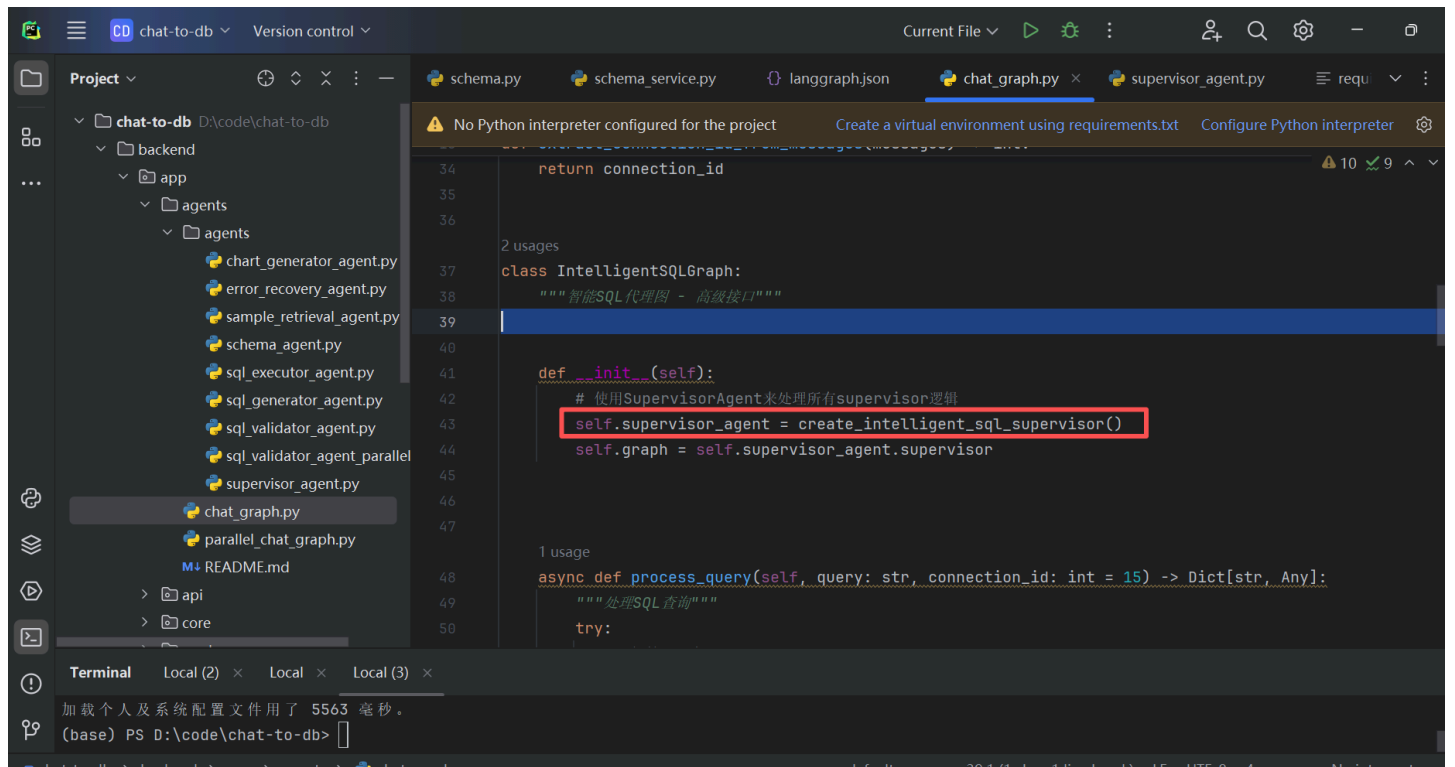
人机交互: <https://docs.langchain.com/oss/python/langchain/human-in-the-loop>

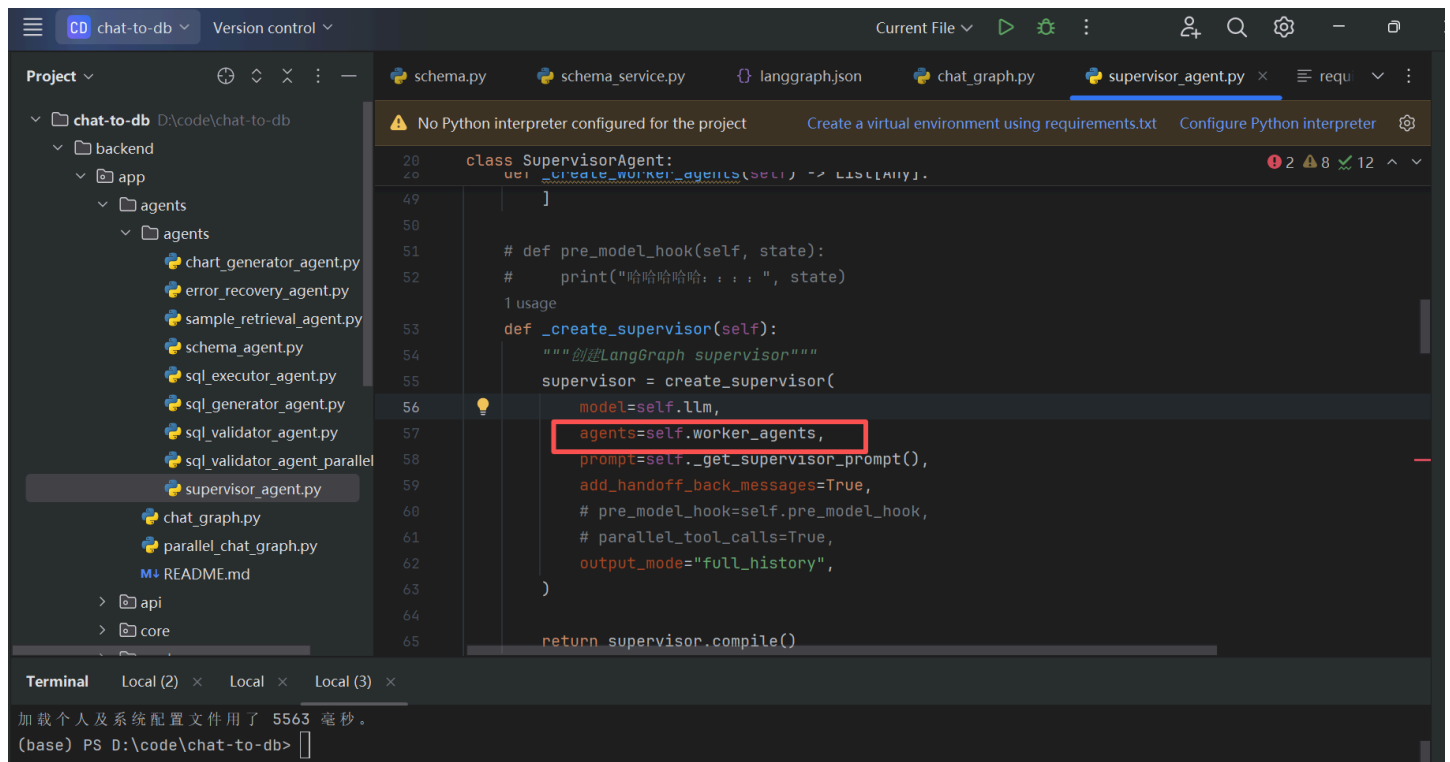
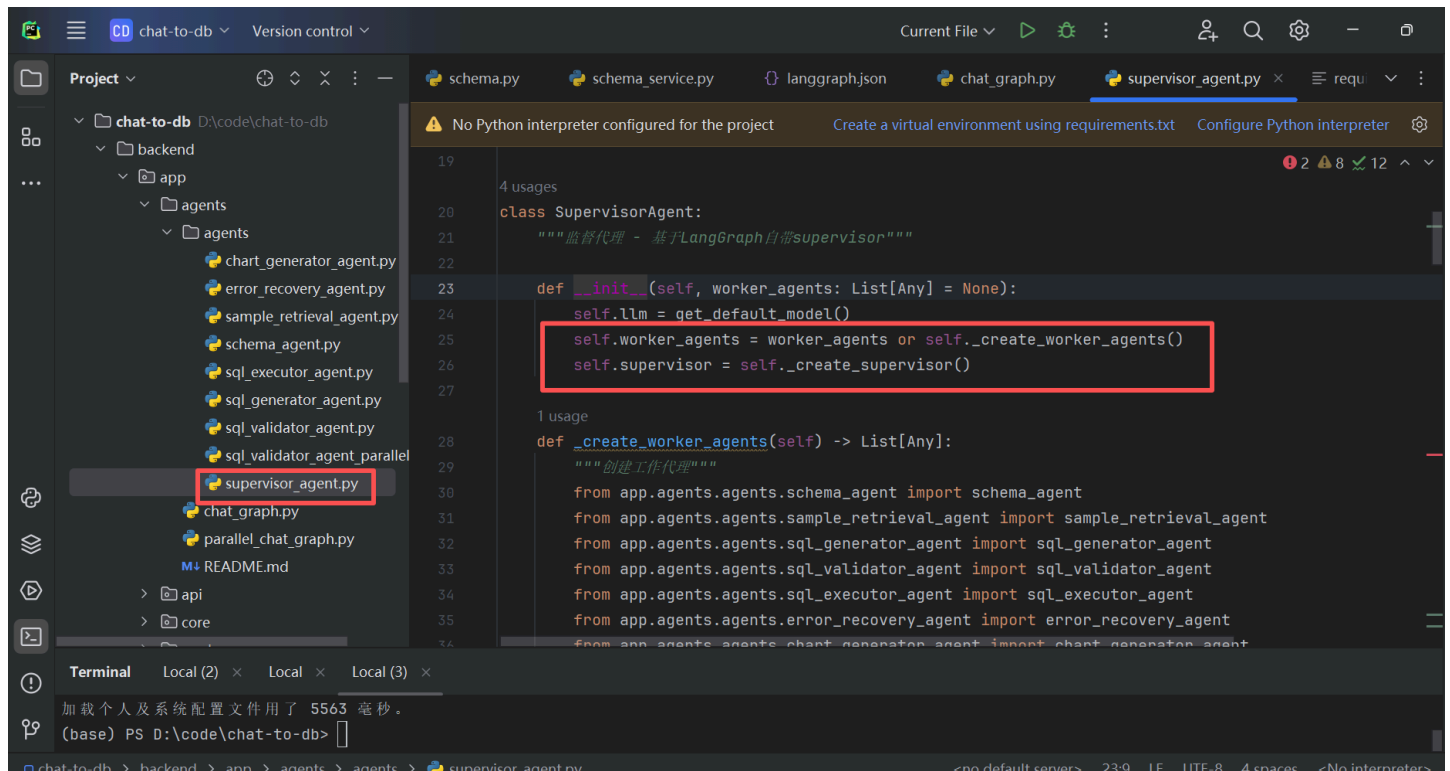
5.3 多智能体核心方案设计与代码落地

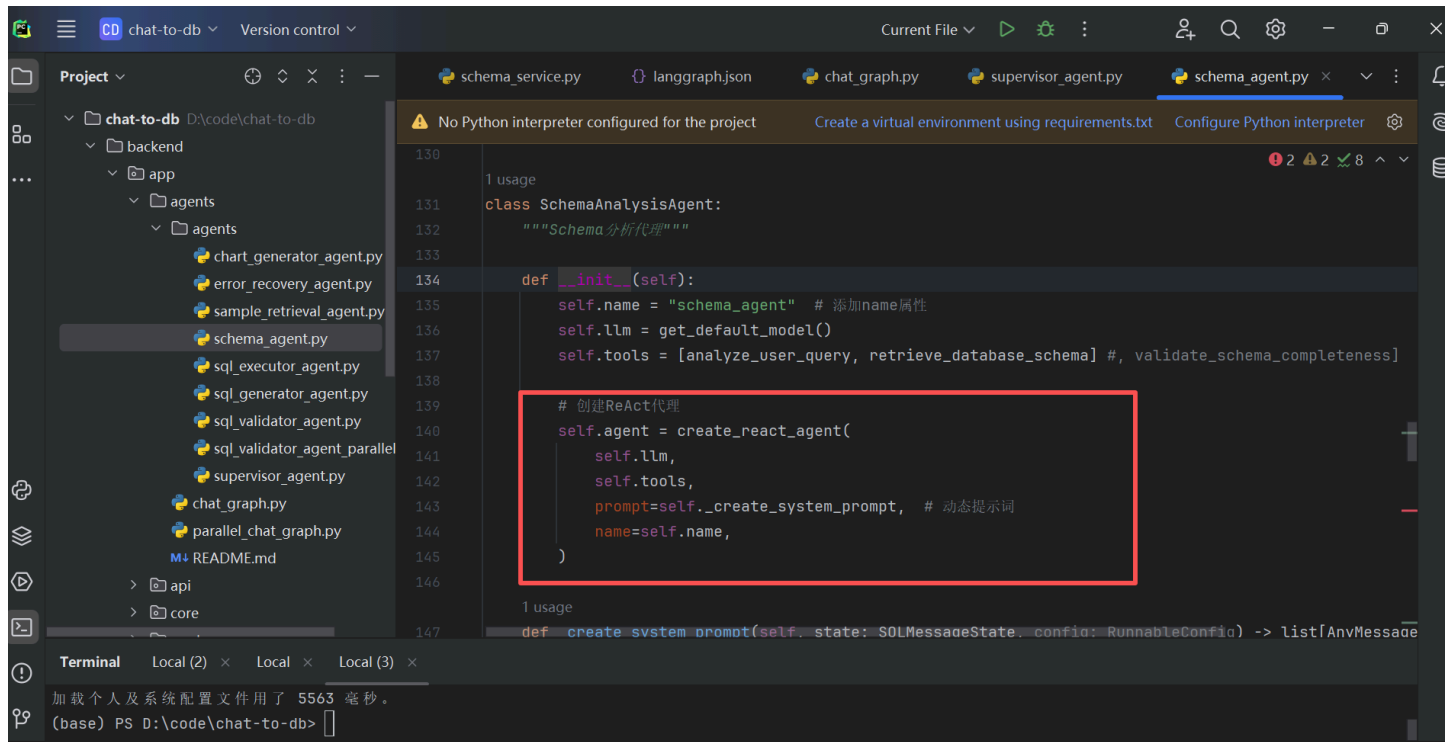
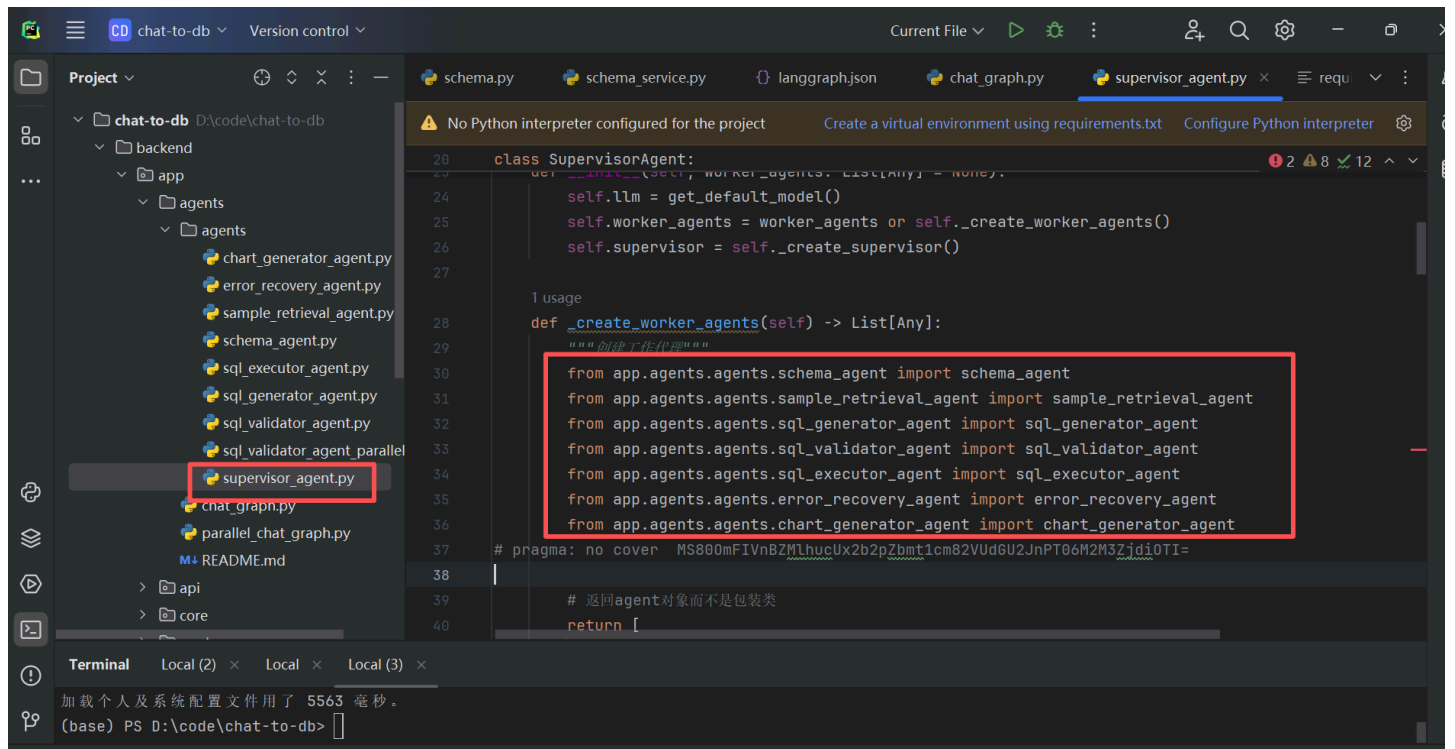
开源项目: <https://github.com/langchain-ai/agent-chat-ui>





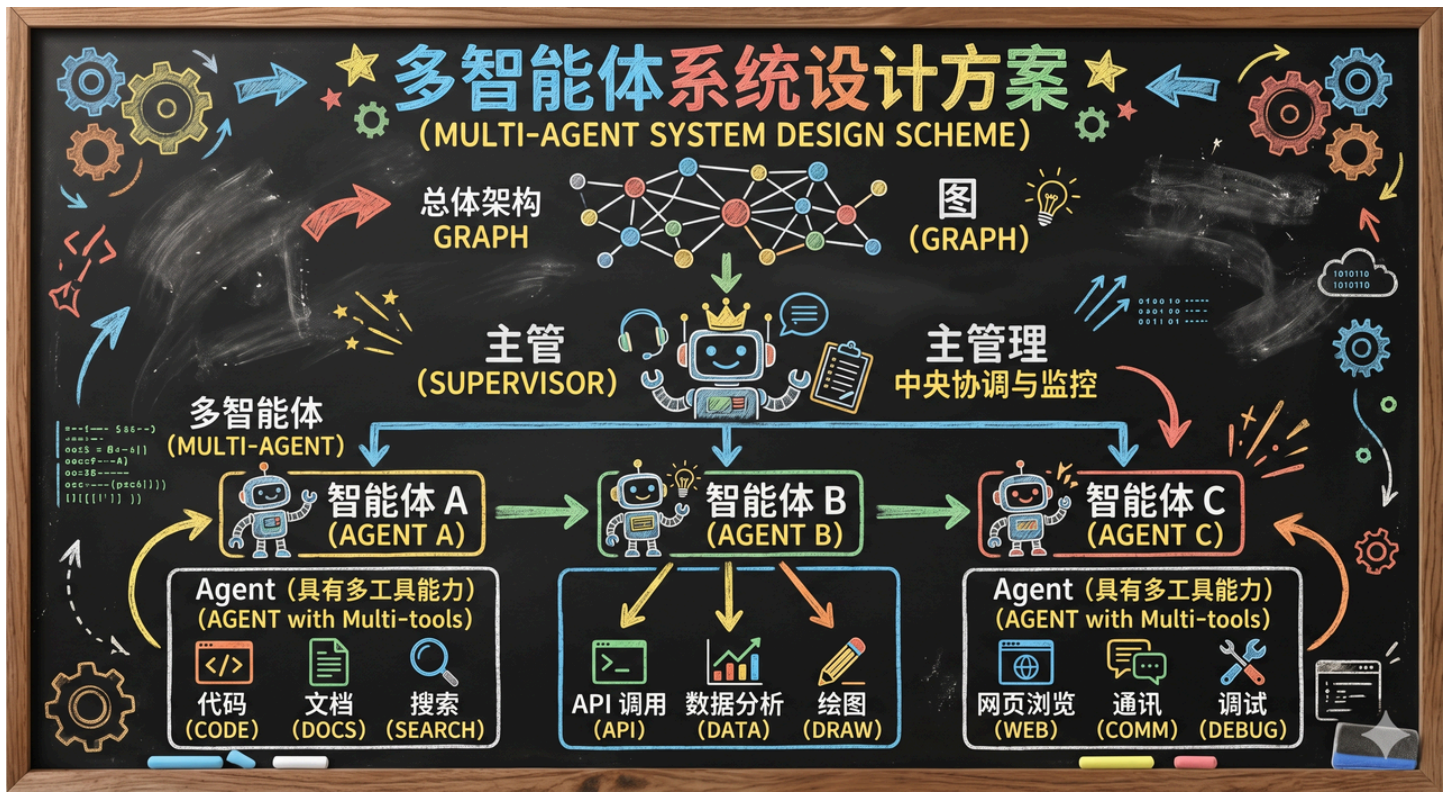






多智能体设计方案

Graph -> Supervisor Agent -> Multi Agent (Agent1/Agent2/...) -> Tools (工具1/工具2/..)



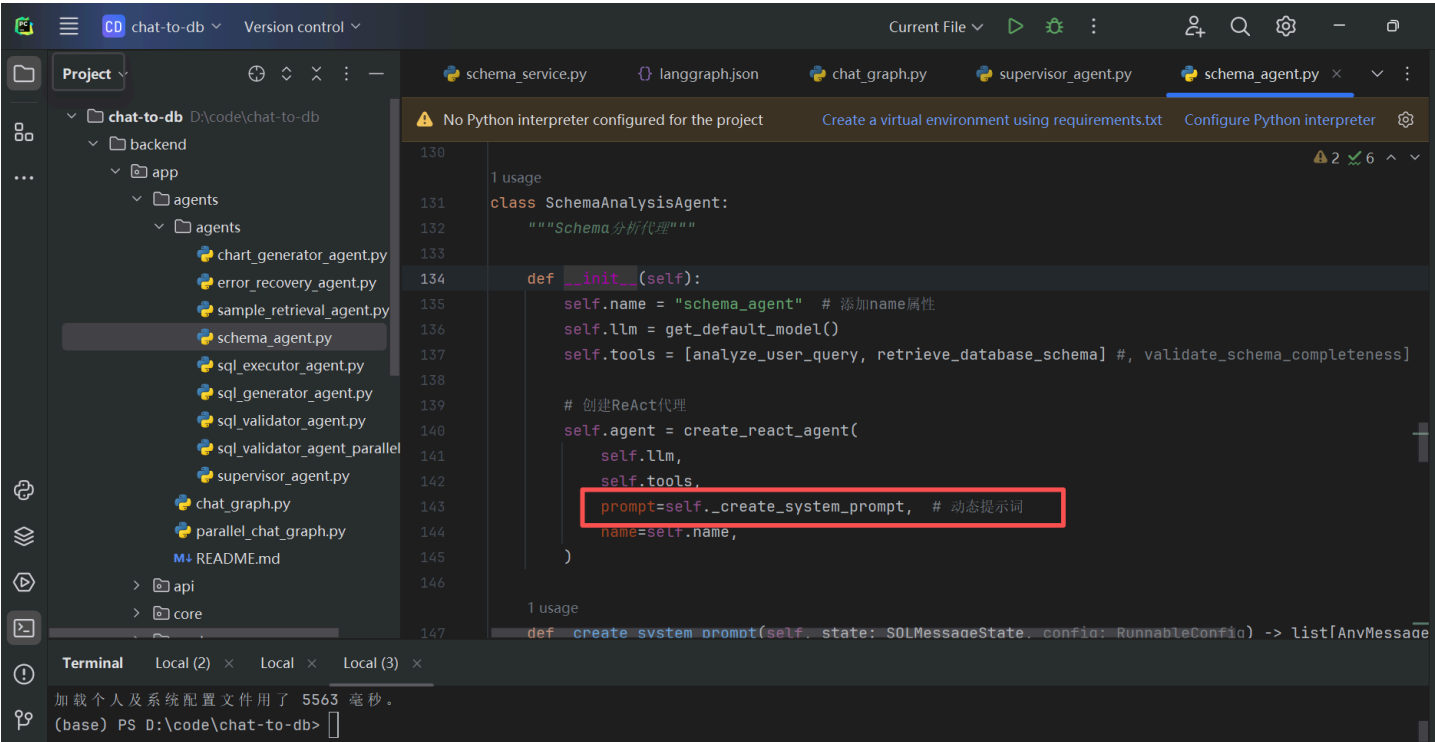
5.4 意图识别智能体设计与代码落地

```
20  
21  
22 1 usage  
23 @tool  
24 def analyze_user_query(query: str) -> Dict[str, Any]:  
25     """  
26     分析用户的自然语言查询，提取关键实体和意图  
27  
28     Args:  
29         query: 用户的自然语言查询  
30  
31     Returns:  
32         包含实体、关系和查询意图的分析结果  
33     """  
34     try:  
35         analysis = analyze_query_with_llm(query)  
36         return {  
37             "success": True,  
38             "analysis": analysis  
39         }
```

工具名称	核心功能	输入参数	返回结果（字典）
analyze_user_query	分析用户自然语言查询，提取查询中的 关键实体、关系和查询意图	query: 用户的自然语言查询字符串	1. 成功: {"success": True, "analysis": 分析结果}2. 失败: {"success": False, "error": 错误信息}
retrieve_database_schema	根据用户查询，从指定数据库中获取 相关的表结构信息和字段值映射	query: 用户查询 connection_id: 数据库连接 ID	1. 成功: {"success": True, "schema_context": 表结构, "value_mappings": 值映射}2. 失败: {"success": False, "error": 错误信息}
validate_schema_completeness	验证获取的数据库模式信息 是否完整 ，检查缺失的实体并给出优化建议	schema_info: 数据库模式信息 query_analysis: 查询分析结果	1. 成功: {"success": True, "is_complete": 是否完整, "missing_entities": 缺失实体列表, "suggestions": 建议列表}2. 失败: {"success": False, "error": 错误信息}

- 1) 获取到用户意图的实体和关系：
- 2) 获取到neo4j -> 获取到所有的表（1000），表名，表的id，表的描述
- 3) 借助大模型（能力一定要强），根据用户的意图识别出来，里面相关的表，3张表（表名，表的id，表的描述）
- 4) 借助于关系找到3张表的关联表，8张表
- 5) 借助大模型（能力一定要强），再次判断，把没有用的去了。剩余5张表
- 6) 5张表（表名，表的id，表的描述），去对应的找到列的信息。拼接出来完整的表结构的信息。

5.5 意图识别智能体动态提示词设计



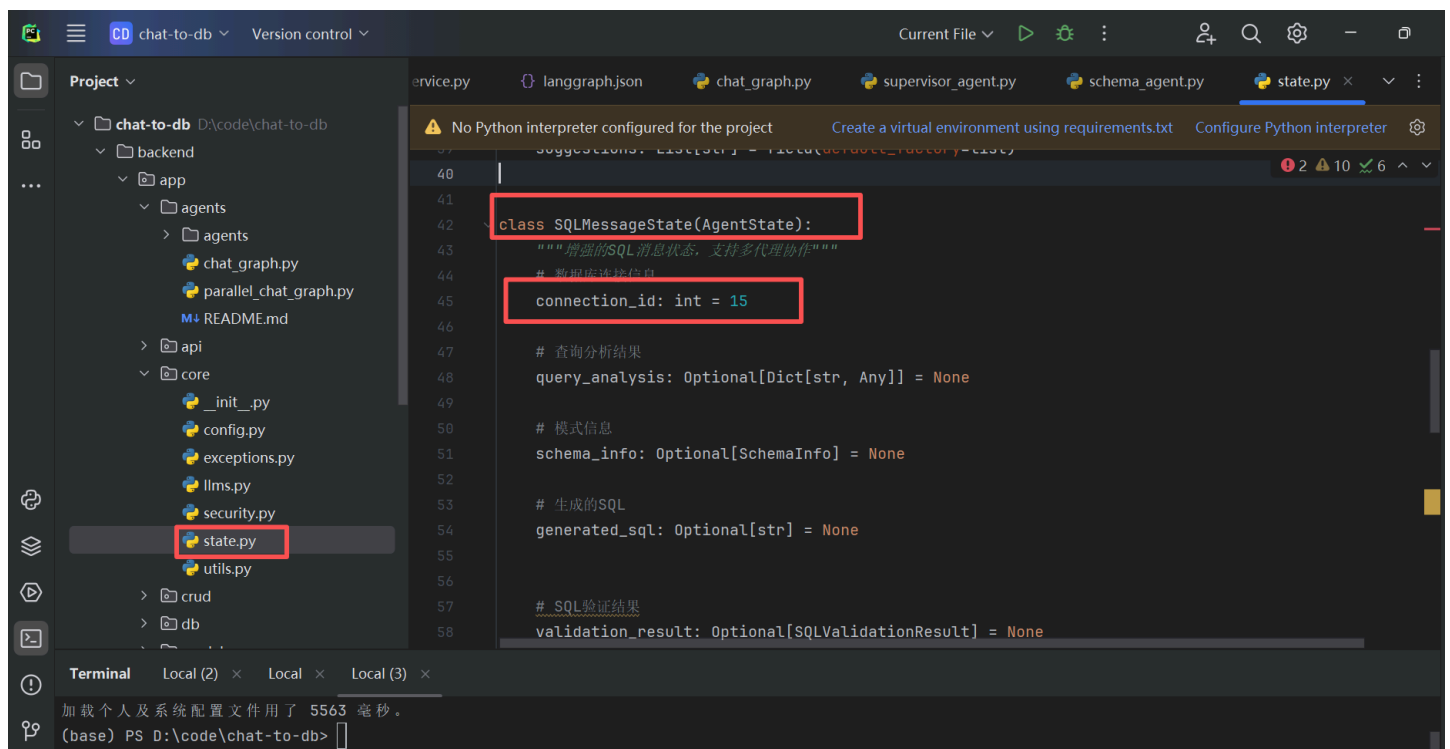
前端返回Supervisor Agent的State信息

代码块

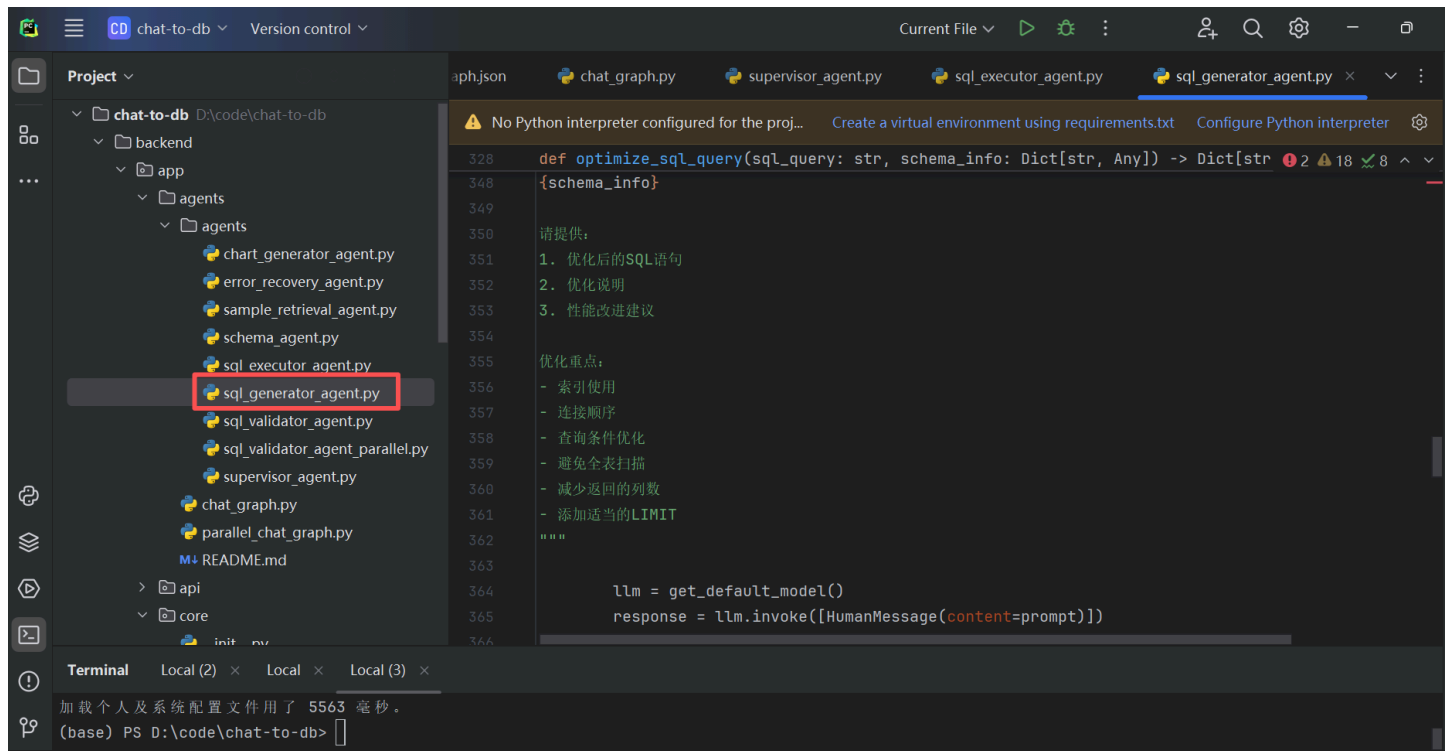
```
1  {'messages': [HumanMessage(content=[{'type': 'text', 'text': '你好'}], additional_kwargs={'connection_id': 15}, response_metadata={}, id='db51c2d1-c74a-4ab1-8ded-b73cefb39ecf')], 'remaining_steps': 199)}
```

代码块

```
1  # 这个是自动的会通过Supervisor传递给子Agent,  
2  HumanMessage(content=[{'type': 'text', 'text': '你好'}])  
3  # 附加的信息是不传的, 但是我们的子Agent需要到这个connection_id的参数  
4  additional_kwargs={'connection_id': 15}  
5  # 所以我们需要把connection_id取出来, 封装到System Message里面, 放到HumanMessage会影响  
   前端的消息展示。  
6
```



5.6 SQL生成智能体设计与代码落地



序号	工具/方法名	类型	功能描述	输入参数	输出结果	核心逻辑要点
1	generate_sql_query	工具函数	根据用户查询和模式信息生成基础SQL语句	user_query , schema_info , value_mappings , db_type , sample_qa_pairs	包含 success , sql_query , context_used 的字典	构建上下文+样本参考→调用LLM→提取并清理SQL代码块
2	generate_sql_with_samples	工具函数	基于高质量样本生成增强型SQL (优先使用)	user_query , schema_info , sample_qa_pairs , value_mappings	包含 success , sql_query , samples_used , best_sample_score , sample_analysis 的字典	过滤低质量样本(阈值0.6)→排序选Top2→构建样本分析→LLM生成
3	analyze_sql_optimization_need	工具函数	分析SQL是否需要性能优化	sql_query , schema_info	包含 needs_optimization , optimization_score , indicators ,	7项指标检测(多JOIN/子查询/无WHERE/SELECT*/多表/复杂函数/无LIMIT)→得分≥3建议优化

					reason 的字典	
4	optimize_sql_query	工具函数	调用LLM优化SQL查询性能	sql_query , schema_info	包含 success , optimized_sql , original_sql 的字典	构建优化提示(索引/连接顺序/条件优化/避免全表扫描)→LLM生成优化方案
5	explain_sql_query	工具函数	解释SQL查询逻辑和执行计划	sql_query	包含 success , explanation , sql_query 的字典	构建解释提示→调用LLM→返回查询逻辑/执行步骤/性能瓶颈/结果集预期
6	_get_optimization_reason	辅助函数	生成优化建议的文本原因	indicators , needs_optimization	优化原因描述字符串	遍历指标布尔值→拼接具体原因→返回建议文本
7	_create_system_prompt	类方法	创建SQL生成代理的系统提示词	无	系统提示字符串	定义角色定位+智能工作流程(样本优先→基础生成→解释)+SQL生成原则+样本利用策略
8	process	类核心方法	编排SQL生成完整流程（入口方法）	state: SQLMessageState	包含 messages , generated_sql , current_stage 的字典	提取用户查询/模式信息/样本→构建输入消息→调用ReAct代理→提取SQL→更新状态
9	_extract_schema_from_messages	辅助方法	从代理消息中解析模式信息	messages	包含提取结果的字典	遍历消息→检测含"schema"关键字的内容→返回结构化信息
10	_extract_sql_from_result	辅助方法	从代理结果消息中提取SQL语句	result	SQL字符串	遍历结果消息→检测含"SELECT"的行→提取首行SQL

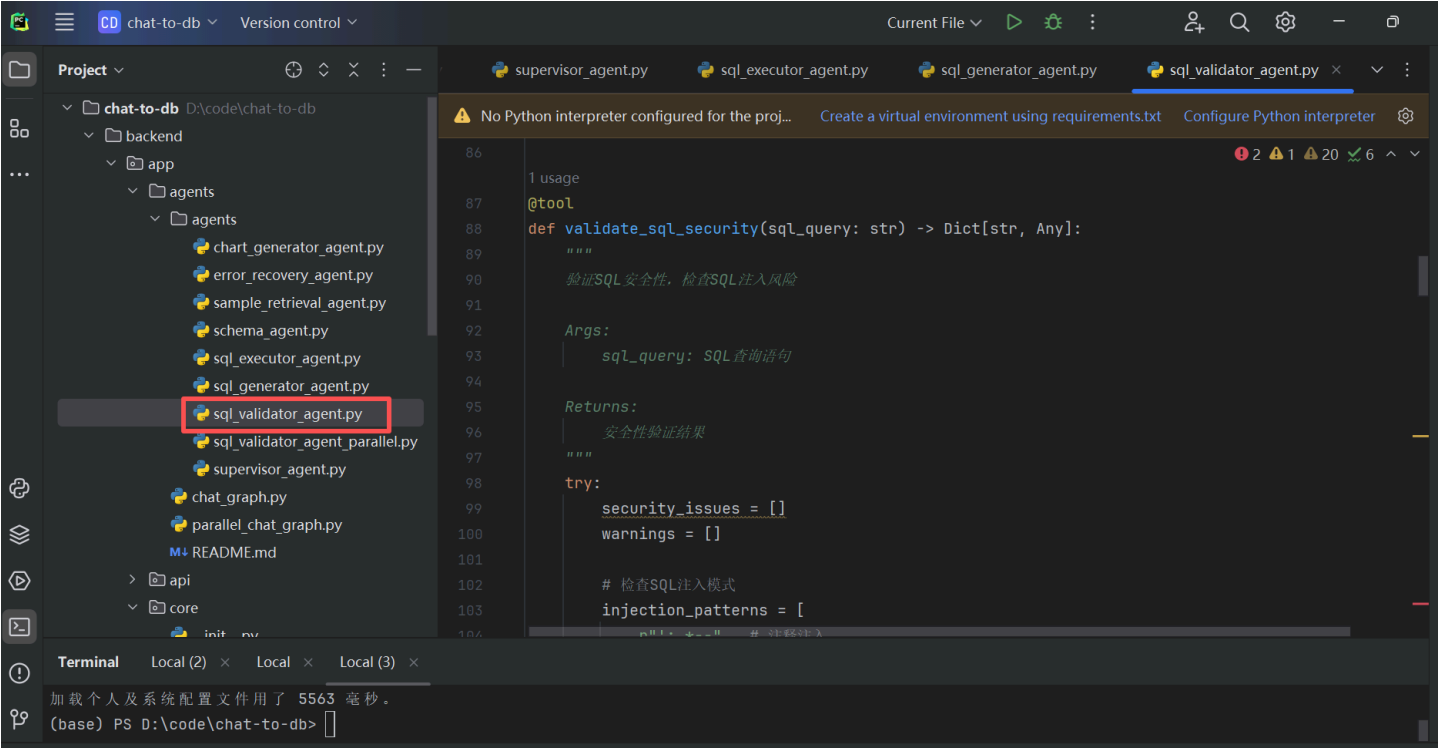
generate_sql_query, generate_sql_with_samples, explain_sql_query

获取到了：表/字段信息/映射 -> 构建SQL语句。

generate_sql_with_samples

样本/表/字段信息/映射信息 -> SQL

5.7 SQL校验智能体设计与代码落地



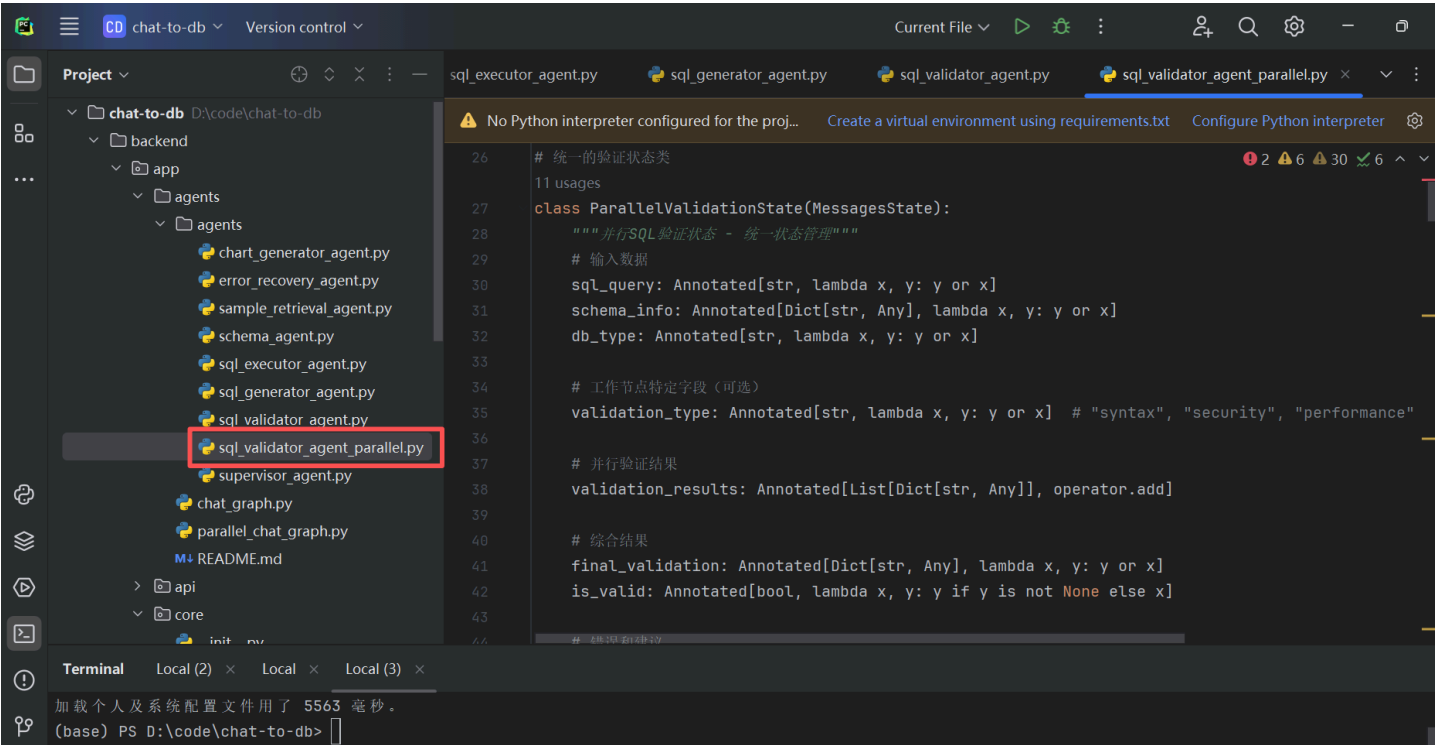
工具方法	功能描述	输入参数	返回值
validate_sql_syntax	验证SQL语法正确性：使用sqlparse解析SQL，检查括号/引号匹配、是否包含SELECT语句、危险操作关键词（DROP/DELETE/UPDATE/INSERT/ALTER等），并建议添加LIMIT子句	sql_query: str , db_type: str = "mysql"	Dict[str, Any] （含 success / is_valid / errors / warnings ）
validate_sql_security	验证SQL安全性，检查SQL注入风险：检测注释注入、UNION注入、逻辑注入（or 1=1 / and 1=1）、存储过程调用等模式，识别字符串拼接和未转义用户输入	sql_query: str	Dict[str, Any] （含 success / is_secure / security_issues / warnings ）

validate_sql_performance	验证SQL性能，识别潜在的性能问题：检测SELECT*、缺失WHERE/LIMIT、CROSS JOIN产生笛卡尔积、过多子查询、ORDER BY无LIMIT、前导通配符LIKE导致索引失效等	sql_query: str , schema_info: Dict[str, Any] = None	Dict[str, Any] (含 success / performance_score / performance_issues / suggestions)
fix_sql_issues	尝试修复SQL中的问题：针对括号不匹配自动补全、缺失SELECT语句时添加子查询包装、未添加LIMIT时追加 LIMIT 100	sql_query: str , validation_errors: List[str]	Dict[str, Any] (含 success / fixed_sql / fixes_applied / original_sql)
human_feedback	需要用户参与解决SQL出现的问题：预留人工介入节点，用于处理自动化工具无法修复的复杂场景（当前为空实现）	state: SQLAlchemyState	None （预留接口）

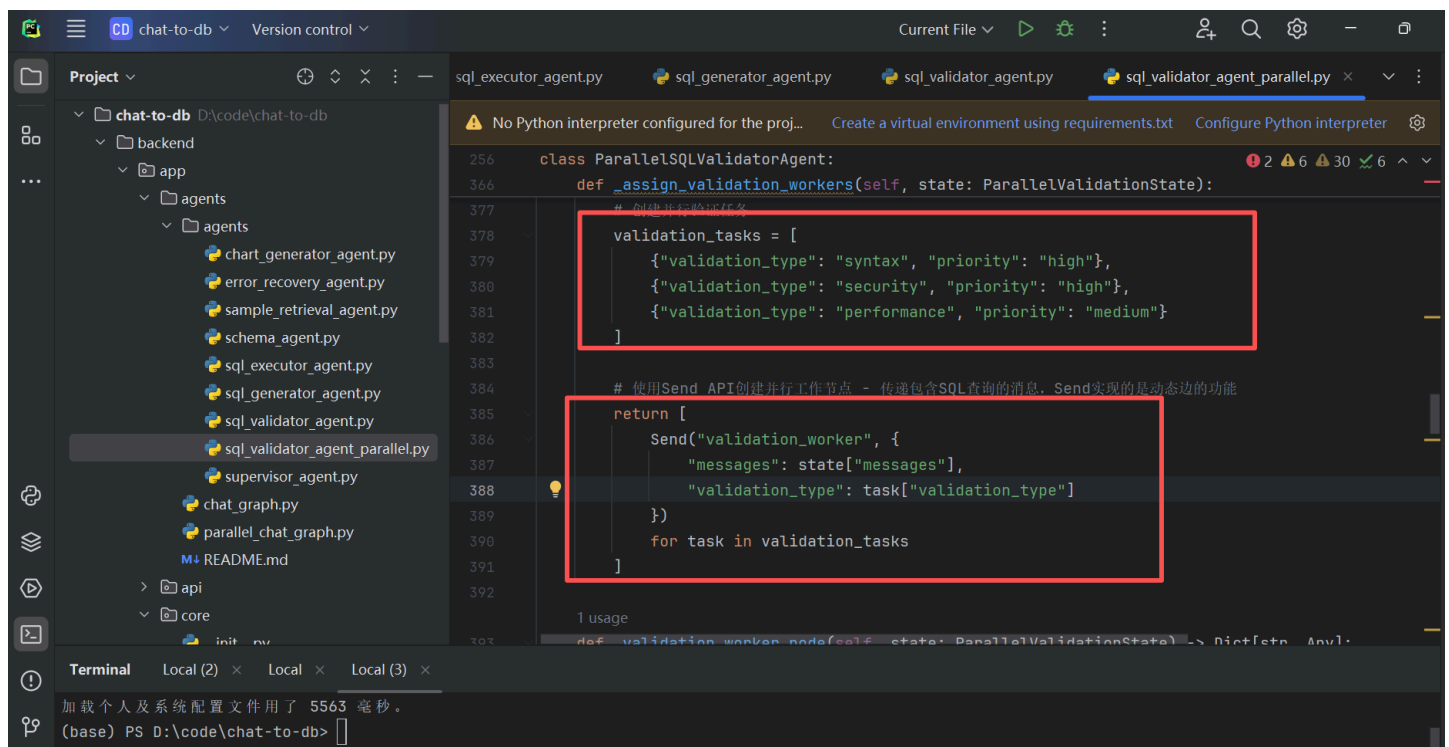
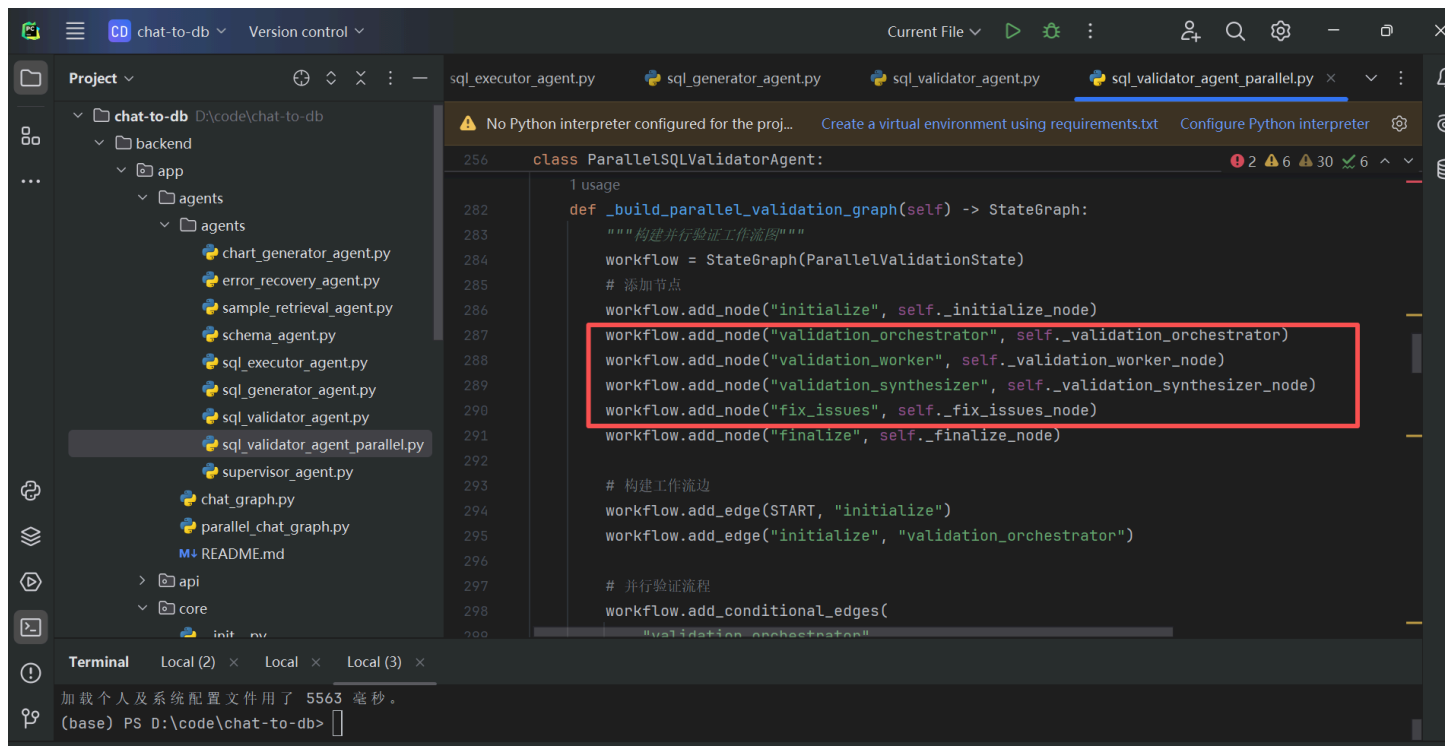
大家可以根据MySQL的explain功能，去做SQL的性能分析

错误修复这儿，修改一下逻辑，让大模型帮忙重新生成SQL（语法错误）

并行校验智能体设计与代码落地



通过图的方式并行的执行。

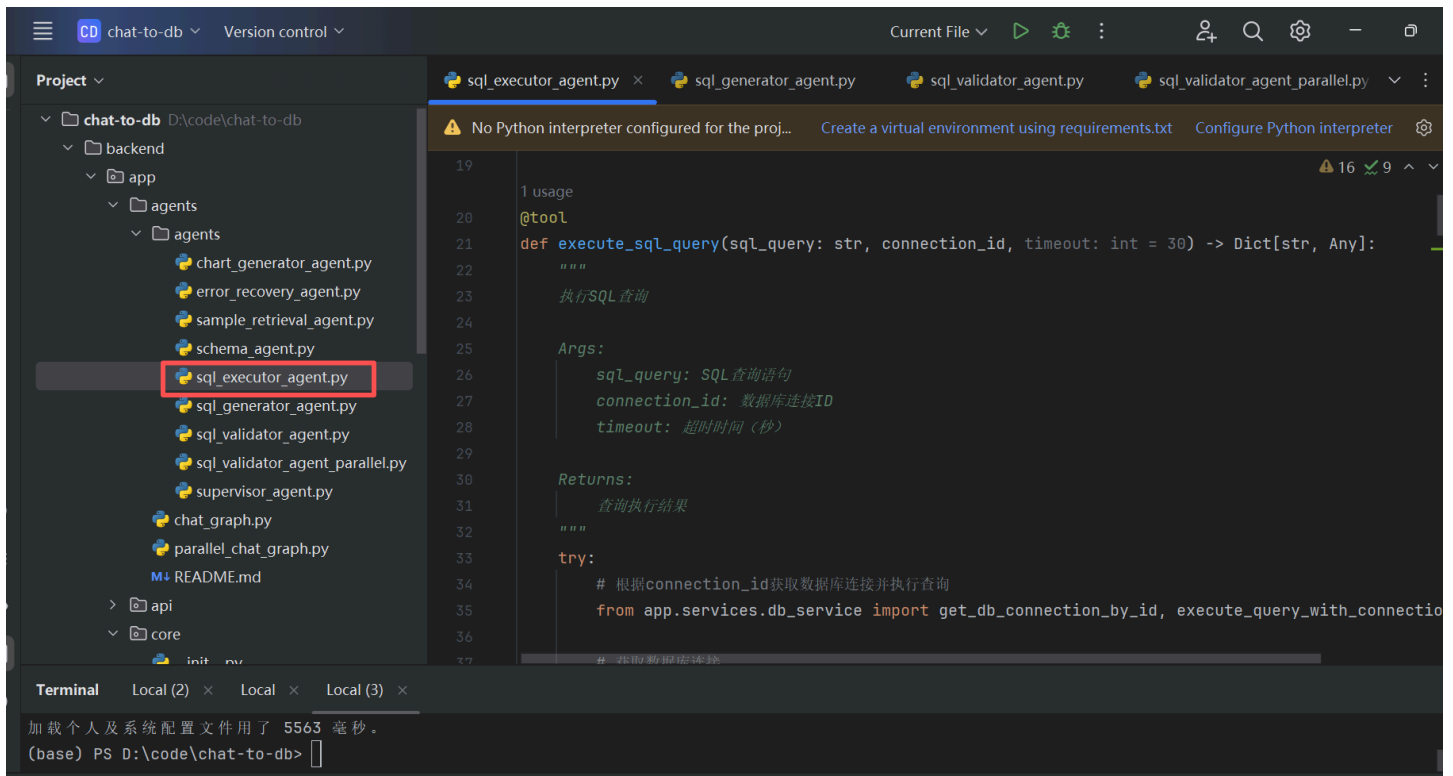


并行执行: <https://docs.langchain.com/oss/javascript/langgraph/graph-api#send>

增加人机交互的逻辑, 人工评审

<https://docs.langchain.com/oss/javascript/langgraph/interrupts#stream-with-human-in-the-loop-hitl-interrupts>

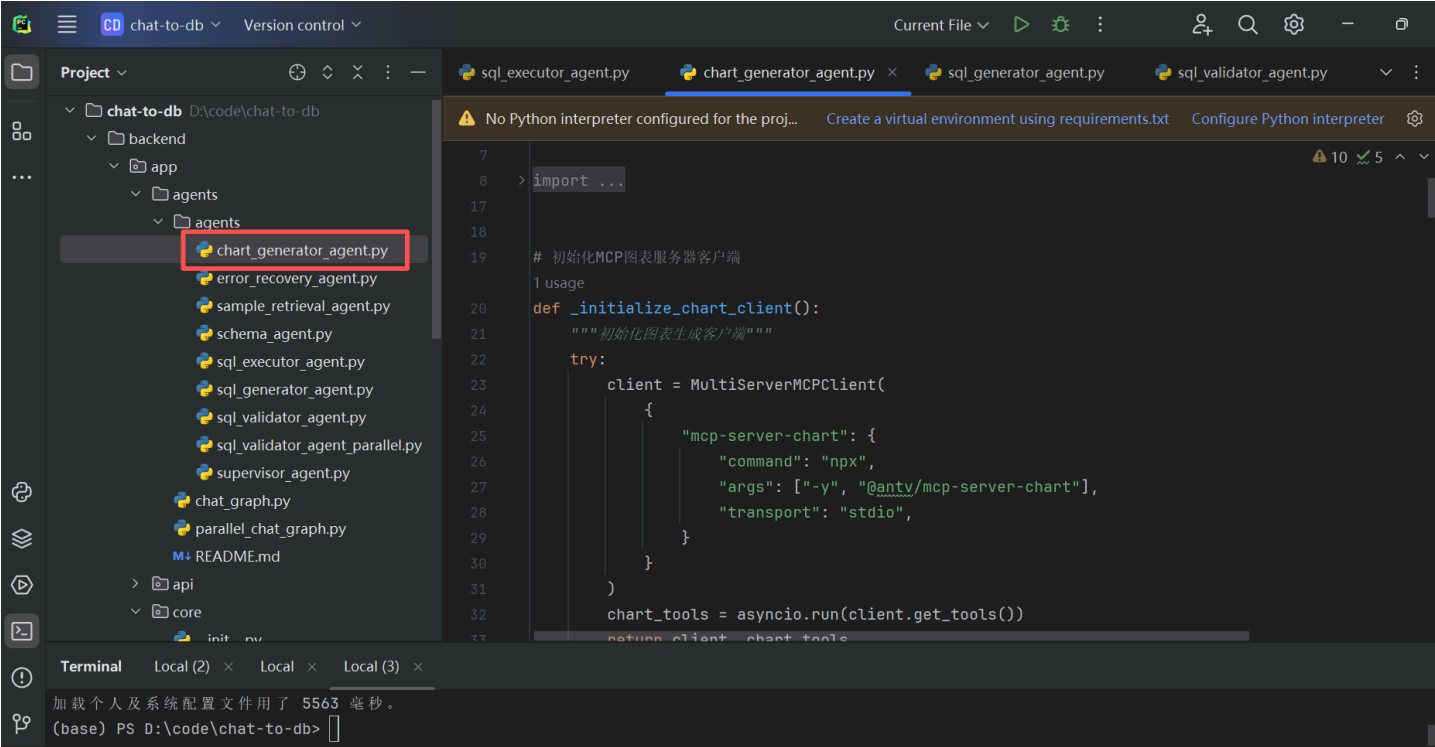
5.8 SQL执行智能体设计与代码落地



序号	工具方法名	功能描述	输入参数	核心逻辑
1	execute_sql_query	安全执行SQL查询并返回结构化结果	sql_query (SQL语句), connection_id (连接ID), timeout (超时秒数, 默认30)	1. 通过 connection_id 获取数据库连接 2. 调用 execute_query_with_connection 执行查询 3. 将结果封装为标准字典格式 4. 异常时返回错误信息
2	analyze_query_performance	分析SQL查询执行性能并给出优化建议	sql_query (SQL语句), execution_result (执行结果字典)	1. 提取执行时间和影响行数 2. 按阈值评定性能等级 (>5秒poor, >2秒fair, >1秒good) 3. 根据耗时和行数生成优化建议 (如加索引、分页等)

3	<code>format_query_results</code>	将查询结果格式化为多种输出形式	<code>execution_result</code> (执行结果字典), <code>format_type</code> (格式类型: table/json/csv, 默认table)	1. 校验执行是否成功 2. 按类型转换: - <code>table</code> : 生成Markdown风格表格 (限制前10行) - <code>json</code> : 转换为字典列表 - <code>csv</code> : 生成逗号分隔文本 3. 异常时返回错误信息
---	-----------------------------------	-----------------	---	--

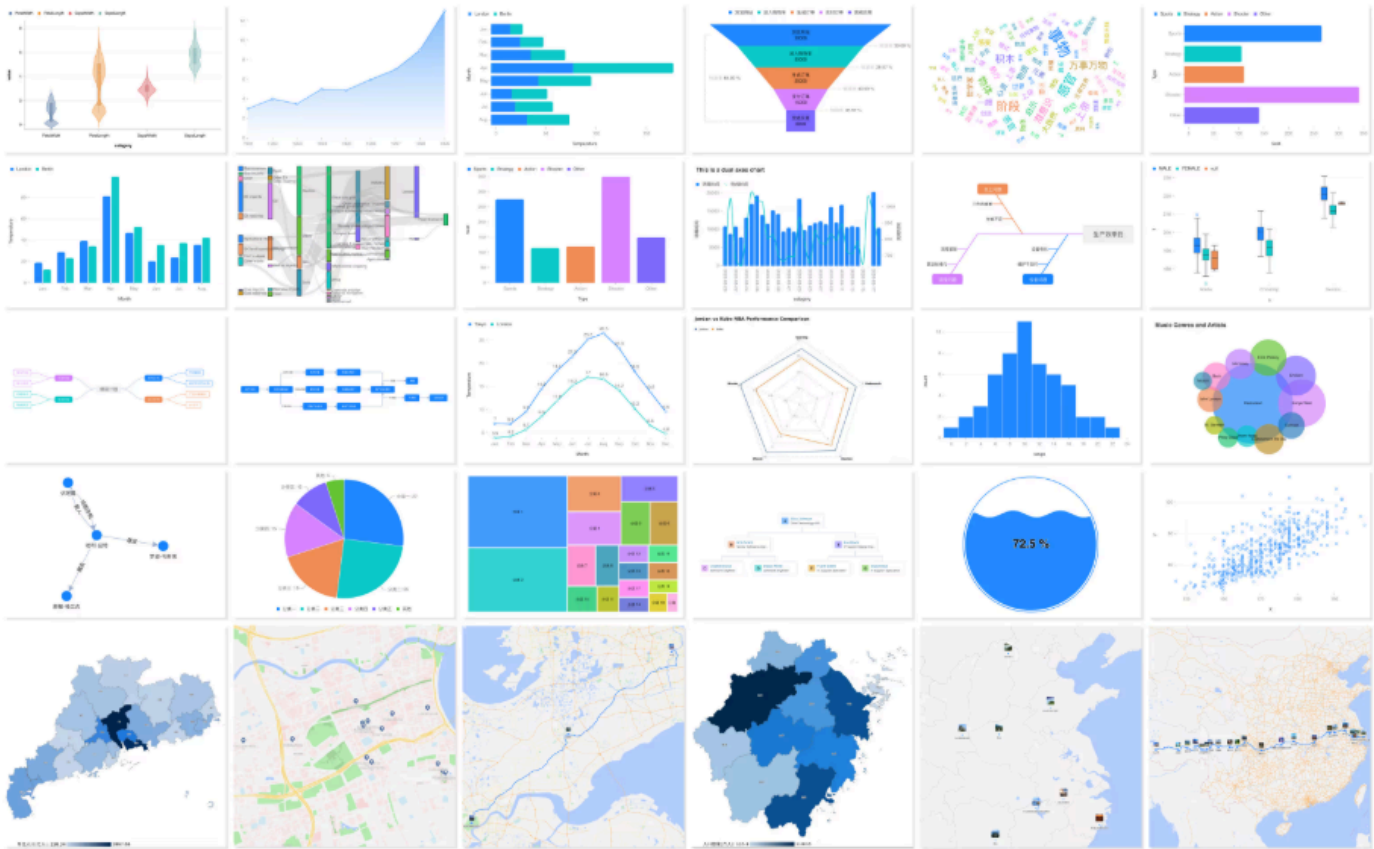
5.9 图表生成智能体设计与代码落地



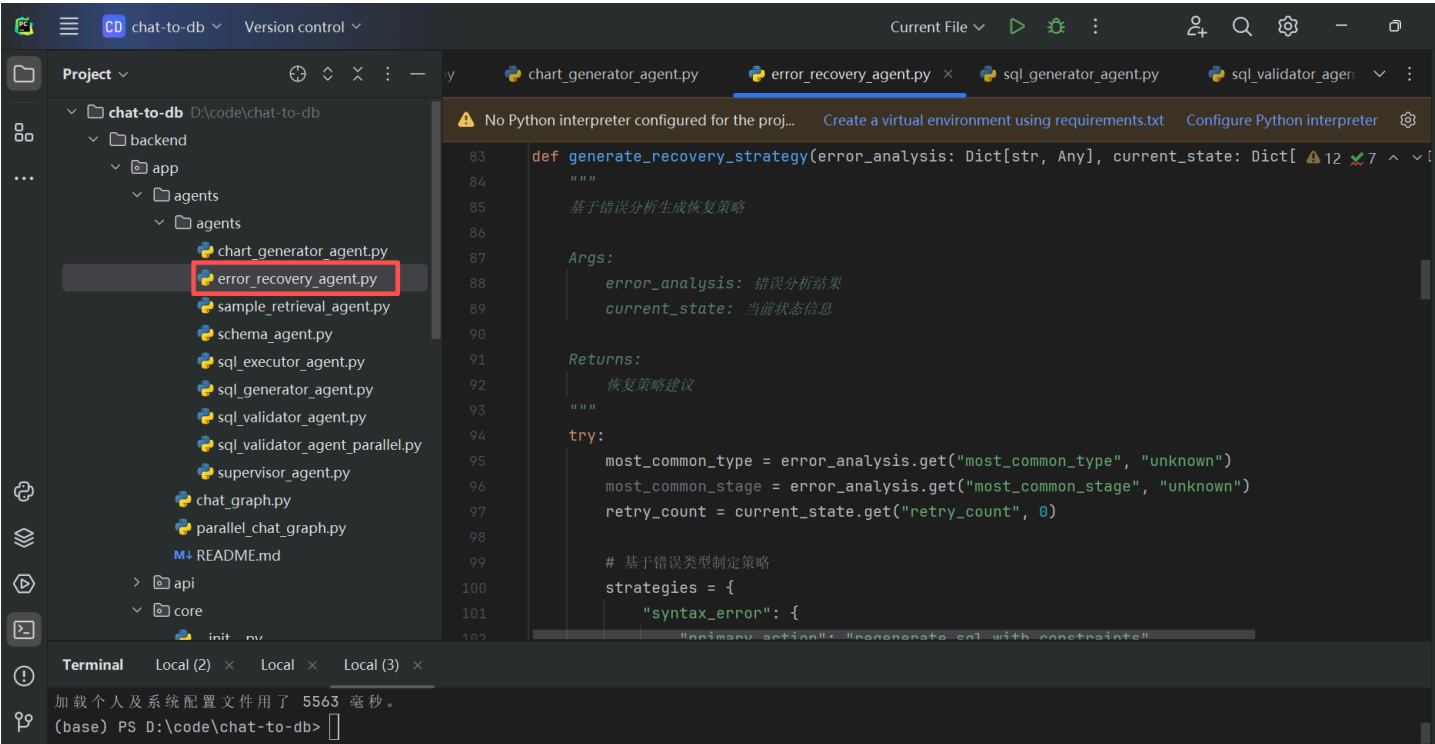
工具方法	类型	功能描述	核心逻辑
<code>analyze_data_for_chart</code>	@tool 装饰器方法	分析数据特征，确定最适合的图表类型	1. 校验数据格式与空值 2. 遍历首行数据，按 <code>int/float / str</code> 分类类型 (含简单日期关键词检测) 3. 调用 <code>_recommend_chart_type</code> 进行图表推荐
<code>_recommend_chart_type</code>	内部私有函数	图表类型推荐的核心逻辑	1. 基于查询上下文关键词匹配 (趋势→折线、比较→柱状、占比→饼图) 2. 降级默认逻辑: 2列数据且行数≤10→饼图, >10→柱状

			图；多数值列→散点图；复杂结构→表格
<code>generate_chart_config</code>	<code>@tool</code> 装饰器方法	生成图表配置，将数据与参数转化为具体图表渲染配置	1. 构建基础配置 (type/data/title/width/height) < 2. 按图表类型注入特定字段： bar→xField/yField/color； line→xField/yField/smooth； pie→angleField/colorField； scatter→xField/yField/size
<code>should_generate_chart</code>	<code>@tool</code> 装饰器方法	判断是否需要生成图表，作为可视化决策的入口	1. 数据格式与空值校验 2. 数据量边界检查 (<<2行或>1000行则拒绝)< 3. 查询意图检测：遍历可视化关键词列表（图表/chart/趋势/分布/占比等）< 4. 数据类型检查：首行是否存在数值型数据 5. 综合判定：用户有可视化意图 或 (有数值数据且列数≥2) → 建议生成
<code>generate_chart</code>	<code>ChartGeneratorAgent</code> 类方法	异步生成图表的入口方法，调用 ReAct 代理执行完整流程	1. 调用 <code>self.agent.ainvoke(state)</code> 触发 ReAct 代理 2. 异常时记录错误到 <code>state["error_history"]</code> ，切换阶段为 <code>error_recovery</code> ，返回 AIMessage 错误提示

MCP工具地址：<https://github.com/antvis/mcp-server-chart>



5.10 错误回复智能体设计与代码落地



工具方法	所属类/模块	功能描述	核心能力	输入参数	返回结果
	独立工具函数				

<code>analyze_error_pattern</code>		分析错误模式，识别重复错误和根本原因	统计错误类型（语法/连接/权限/超时/未知）、统计错误阶段、识别高频错误模式	<code>error_history</code> ：错误历史记录列表	错误模式分析结果（含类型分布、阶段分布、最常见类型/阶段、总错误数）
<code>generate_recovery_strategy</code>	独立工具函数	基于错误分析生成恢复策略	根据错误类型匹配预设策略（语法错误→重新生成SQL、连接错误→检查连接配置、权限错误→修改查询范围、超时错误→优化性能）、根据重试次数调整策略置信度、生成具体恢复步骤	<code>error_analysis</code> ：错误分析结果； <code>current_state</code> ：当前状态信息（含重试次数）	恢复策略建议（含主/次行动、描述、自动修复可能性、置信度、具体步骤、预估成功率）
<code>auto_fix_sql_error</code>	独立工具函数	自动修复SQL错误	修复语法错误（补分号、修正关键字大小写、修复未闭合引号）、修正表名/字段名（基于schema模糊匹配）、性能优化（添加LIMIT）、权限降级（SELECT * → SELECT id, name）	<code>sql_query</code> ：有问题的SQL； <code>error_message</code> ：错误消息； <code>schema_info</code> ：模式信息（可选）	修复结果（含原始SQL、修复后SQL、应用的修复项、是否需要人工审核）
<code>recover</code>	<code>ErrorRecoveryAgent</code> 类方法	执行完整的错误恢复流程	协调三大工具完成端到端恢复：调用分析→生成策略→尝试修复；管理重试计数和阶段流转；更新SQLMessageState状态	<code>state</code> ：SQLMessageState（含错误历史、当前SQL、schema信息）	恢复结果（含成功标志、恢复状态、修复后SQL、下一流程阶段）
<code>_parse_recovery_result</code>	<code>ErrorRecoveryAgent</code> 类方法	解析LLM代理返回的恢复结果	从ReAct代理的消息列表中提取	<code>result</code> ：ReAct代理返回	结构化恢复结果（含修复成功标

	法		关键信息：判断是否修复成功、提取修复后的SQL语句、确定下一阶段	结果； state：当前状态	志、修复SQL、恢复策略、下一阶段、原始消息)
_create_system_prompt	ErrorRecoveryAgent 类方法	创建系统提示词	定义错误恢复专家的角色定位、恢复流程规范（分析→策略→修复）、恢复原则（优先自动修复、避免重复错误、评估成功率）	无	系统提示字符串

6. 智能问数项目总结

6.1 人机交互？

SQL -> Executor SQL ? -> 本来懂业务/SQL -> 数据分析师，产品经理，程序员（SQL，业务）

SQL -> Executor SQL ? -> 如果用户是小白，不希望交互，那就不用交互

人机交互：<https://docs.langchain.com/oss/python/langchain/human-in-the-loop>

Vibe Coding 课程链接：<https://www.bilibili.com/cheese/play/ss905865514>

6.2 准确率？

ChatBI Text2SQL -> 思路 -> 代码不关键 -> vibe coding技术

准确率也不高？

用户提示词(Query改造?)/词语的映射（全还是不全?）/表的信息|字段的信息(描述准确吗?)/样本的信息（样本的质量怎么样?）大模型(模型能力不够，deepseek3 deepseek4) -> 构建SQL -> 执行SQL -> 可视化的结果（MCP）

Case by case 解决问题！

